

Design Patterns & Architectural Patterns for Front-End Engineers

Sudeep Parchure | AI Assisted | May 2026 | Angular Solutions Architect

Source code and working examples: github.com/sudeep31/angularTodo — documents folder

Table of Contents

- [1. Introduction — Why Patterns Matter](#)
- [2. Design Patterns — GoF & Front-End](#)
 - [2.1 Creational Patterns](#)
 - [2.2 Structural Patterns](#)
 - [2.3 Behavioural Patterns](#)
- [3. Architectural Patterns](#)
 - [3.1 Micro-Frontend \(MFE\)](#)
 - [3.2 Atomic Design](#)
 - [3.3 Backend for Frontend \(BFF\)](#)
 - [3.4 Saga Pattern](#)
 - [3.5 CQRS & Event Sourcing](#)
 - [3.6 Flux / NgRx \(Redux\)](#)
 - [3.7 Smart / Presentational Components](#)
 - [3.8 Feature-Sliced Design](#)
- [4. Architectural Strategies](#)
 - [4.1 The 12-Factor App — Front-End Context](#)
 - [4.2 The 6-Hour Design Approach](#)
- [5. Tools & Diagram Types](#)
- [6. System Design Case Studies](#)
 - [6.1 Enterprise Banking Application](#)
 - [6.2 AngularJS → Angular 21.2 Migration](#)
 - [6.3 MFE Deployment Topologies for Legacy Migration](#)
- [7. Resources](#)

1. Introduction — Why Patterns Matter

Patterns are **proven, reusable solutions to recurring problems** in software design. They are not copy-paste code — they are vocabulary and blueprints. For a front-end engineer working with Angular 21, patterns answer three questions:

Question	Pattern Category	Example
How do I structure this object or service?	Design Pattern (GoF)	Factory, Singleton, Facade
How do I organise the application?	Architectural Pattern	MFE, Atomic Design, NgRx
How do I make the right strategic decisions at scale?	Architectural Strategy	12-Factor, 6-Hour Design

Angular 21 Context

Angular 21 introduces features that change *how* classic patterns are implemented:

Angular 21 Feature	Pattern Impact
Signals (<code>signal()</code> , <code>computed()</code> , <code>effect()</code>)	Replaces RxJS for local state; changes Observer pattern implementation
<code>input()</code> / <code>output()</code> functions	Replaces <code>@Input()</code> / <code>@Output()</code> decorators
<code>inject()</code> function	Replaces constructor injection; enables functional patterns
Signal-based forms (<code>FormField</code> , <code>linkedSignal</code>)	Replaces reactive forms for many use cases
Standalone components (no <code>NgModule</code>)	Enables Feature-Sliced Design and lazy loading without modules
SSR + hydration (Angular 21)	Affects BFF and state transfer patterns
<code>@defer</code> blocks	Lazy-loading pattern built into templates

Principle: Never apply a pattern just because it exists. Apply it when the problem it solves is present in your codebase.

2. Design Patterns — GoF & Front-End

The Gang of Four (GoF) catalogue defines 23 patterns across three categories. This section maps each relevant pattern to Angular 21, shows how it is implemented, and gives real-world examples.

2.1 Creational Patterns

Creational patterns deal with **how objects are created**, decoupling the creation logic from the consuming code.

Factory Pattern

What it is: A function or class that creates objects without exposing instantiation logic to the caller. The caller asks for a product; the factory decides which concrete class to return.

How it helps: Centralises creation logic, makes swapping implementations easy, and removes `new` calls from consuming code.

Angular 21 implementation:

```

// token-storage.factory.ts
export type StorageType = 'local' | 'session' | 'memory';

export interface TokenStorage {
  get(key: string): string | null;
  set(key: string, value: string): void;
  remove(key: string): void;
}

class LocalTokenStorage implements TokenStorage {
  get(key: string) {
    return localStorage.getItem(key);
  }
  set(key: string, value: string) {
    localStorage.setItem(key, value);
  }
  remove(key: string) {
    localStorage.removeItem(key);
  }
}

class SessionTokenStorage implements TokenStorage {
  get(key: string) {
    return sessionStorage.getItem(key);
  }
  set(key: string, value: string) {
    sessionStorage.setItem(key, value);
  }
  remove(key: string) {
    sessionStorage.removeItem(key);
  }
}

class MemoryTokenStorage implements TokenStorage {
  private store = new Map<string, string>();
  get(key: string) {
    return this.store.get(key) ?? null;
  }
  set(key: string, value: string) {
    this.store.set(key, value);
  }
  remove(key: string) {
    this.store.delete(key);
  }
}

// The Factory
export function createTokenStorage(type: StorageType): TokenStorage {
  switch (type) {
    case 'local':
      return new LocalTokenStorage();
    case 'session':
      return new SessionTokenStorage();
    case 'memory':
      return new MemoryTokenStorage();
  }
}

// Angular DI provider using factory
export const TOKEN_STORAGE = new InjectionToken<TokenStorage>('TOKEN_STORAGE', {
  providedIn: 'root',
  factory: () => createTokenStorage(inject(PLATFORM_ID) === 'browser' ? 'local' : 'memory'),
});

```

Code walkthrough — how this becomes the Factory Pattern:

1. **StorageType and TokenStorage interface** — Define the contract first. Every storage class must implement `get()`, `set()`, and `remove()`. Callers will only ever see this interface — never a concrete class.
2. **LocalStorage, SessionTokenStorage, MemoryTokenStorage** — Three concrete implementations, each wrapping a different browser/server mechanism. Crucially, none of these classes are exported. They are implementation details hidden inside the file.
3. **createTokenStorage(type)** — **this is the Factory function**. It accepts a plain `StorageType` string and returns whichever concrete class matches. The caller never writes `new LocalTokenStorage()`. That decision lives in one place: the factory.
4. **TOKEN_STORAGE injection token** — Angular's DI calls the factory function automatically. It reads `PLATFORM_ID` to pick `'local'` in the browser and `'memory'` on the server (SSR). Any service that injects `TOKEN_STORAGE` gets the right implementation with no platform-detection code of its own.

Pattern conclusion: The interface hides the concrete types. The factory centralises all creation logic. Consumers depend on the `TokenStorage` abstraction — they never import or instantiate a storage class directly. Swapping the implementation (e.g. using an encrypted storage class) means changing one line inside the factory — zero changes anywhere else.

Real-world use: Banking app — SSR uses in-memory storage, browser uses `localStorage`. The factory decides at runtime without the consuming service knowing.

Singleton Pattern

What it is: Ensures a class has only one instance and provides a global access point to it.

How it helps: Shared state (auth tokens, feature flags, app config) lives in one place, preventing inconsistency.

Angular 21 implementation:

Angular's DI system is the Singleton pattern — `providedIn: 'root'` creates exactly one instance across the app.

```
// auth.service.ts — Angular-native singleton
@Injectable({ providedIn: 'root' })
export class AuthService {
  private readonly _user = signal<User | null>(null);
  readonly user = this._user.asReadonly();
  readonly isAuthenticated = computed(() => this._user() !== null);

  login(credentials: Credentials): Observable<void> {
    return this.http
      .post<AuthResponse>('/api/auth/login', credentials)
      .pipe(tap((response) => this._user.set(response.user)));
  }

  logout(): void {
    this._user.set(null);
  }
}
```

Code walkthrough — how this becomes the Singleton Pattern:

1. `@Injectable({ providedIn: 'root' })` — This single line is the Singleton declaration. It tells Angular's DI container to create exactly **one instance** of `AuthService` for the entire application. Angular's injector tree is the Singleton registry — no static variables needed.
2. `private readonly _user = signal<User | null>(null)` — A private writable signal. Only methods inside this class can change the user. This enforces controlled mutation — the core discipline of a Singleton.
3. `readonly user = this._user.asReadonly()` — A public read-only projection. Consumers can read the current user but cannot call `_user.set()` from outside.
4. `readonly isAuthenticated = computed(...)` — Derived state computed from the single source of truth. All consumers that read this signal see the same value because they all reference the same instance.
5. `login()` / `logout()` — The only mutation paths. Because every component injects the **same object**, calling `logout()` in the top nav immediately updates every other component reading `isAuthenticated` or `user`.

Pattern conclusion: `providedIn: 'root'` is Angular's native Singleton mechanism. Every component, service, and interceptor that injects `AuthService` receives the **same object**. There is no `static getInstance()` method, no module-level variable — the DI container manages the lifecycle and guarantees uniqueness.

Anti-pattern warning: Do not implement the classic `getInstance()` static method in Angular. Let the DI container manage the singleton lifecycle.

Real-world use: `AuthService`, `FeatureFlagService`, `AppConfigService` — all singletons that multiple unrelated components query.

Builder Pattern

What it is: Separates the construction of a complex object from its representation, allowing step-by-step construction.

How it helps: Avoids constructor telescoping (10-parameter constructors). Makes complex object creation readable and testable.

Angular 21 implementation:

```
// http-request.builder.ts
export class HttpRequestBuilder<T> {
  private url = '';
  private params = new HttpParams();
  private headers = new HttpHeaders();
  private body: unknown = null;

  withUrl(url: string): this {
    this.url = url;
    return this;
  }

  withParam(key: string, value: string): this {
    this.params = this.params.set(key, value);
    return this;
  }

  withBearerToken(token: string): this {
    this.headers = this.headers.set('Authorization', `Bearer ${token}`);
    return this;
  }

  withBody(body: unknown): this {
    this.body = body;
    return this;
  }

  build(): { url: string; options: object } {
    return {
      url: this.url,
      options: { params: this.params, headers: this.headers, body: this.body },
    };
  }
}

// Usage
const request = new HttpRequestBuilder<Transaction[]>()
  .withUrl('/api/transactions')
  .withParam('accountId', '12345')
  .withParam('pageSize', '50')
  .withBearerToken(inject(AuthService).token())
  .build();
```

Code walkthrough — how this becomes the Builder Pattern:

1. **Private fields** (`url`, `params`, `headers`, `body`) — These are the parts of the product under construction. They are private and not exposed until `build()` is called. Nobody outside the builder can see a half-assembled request.
2. `withUrl()`, `withParam()`, `withBearerToken()`, `withBody()` — Each method sets one property and returns `this`. Returning `this` is the Builder signature move — it is what enables **method chaining**.
3. **Method chaining is self-documenting** — The call chain reads: *"Build a request, set its URL, add two query params, attach a bearer token, then build."* There are no positional arguments to misorder and no optional constructor parameters to guess at.
4. `build()` — The terminal step. It assembles all accumulated parts into the final result object. Only this method produces output. You cannot accidentally use a half-built request because there is no property to read until `build()` is called.
5. **Usage** — The caller never constructs `HttpParams` or `HttpHeaders` directly. The builder hides that complexity and produces a clean `{ url, options }` object.

Pattern conclusion: Without a builder you would pass 6 optional parameters to a function and struggle to remember which position is which. The builder names each step, makes the construction order legible, and enforces a `build()` gate so only complete objects are produced. Adding a new parameter (e.g. a timeout) is adding one `withTimeout()` method — no existing callers break.

Real-world use: Building paginated API requests, constructing chart config objects, assembling form field metadata.

2.2 Structural Patterns

Structural patterns deal with **how objects and classes are composed** to form larger structures.

Facade Pattern

What it is: Provides a simplified interface to a complex subsystem of services, APIs, or libraries.

How it helps: Components talk to one facade instead of six services. The facade owns the orchestration complexity.

Angular 21 implementation:

```
// account-facade.service.ts
@Injectable({ providedIn: 'root' })
export class AccountFacade {
  private readonly accountApi = inject(AccountApiService);
  private readonly transactionApi = inject(TransactionApiService);
  private readonly notificationService = inject(NotificationService);
  private readonly store = inject(Store);

  // Signals exposed to components
  readonly accounts = toSignal(this.store.select(selectAllAccounts), { initialValue: [] });
  readonly isLoading = toSignal(this.store.select(selectAccountsLoading), { initialValue: false });

  loadDashboard(userId: string): void {
    this.store.dispatch(AccountActions.loadAccounts({ userId }));
  }

  async transferFunds(from: string, to: string, amount: number): Promise<void> {
    try {
      await firstValueFrom(this.accountApi.transfer({ from, to, amount }));
      this.notificationService.success(`Transferred £${amount}`);
      this.store.dispatch(AccountActions.reloadAccount({ id: from }));
    } catch (error) {
      this.notificationService.error('Transfer failed - please retry');
      throw error;
    }
  }
}
```

```
// dashboard.component.ts — only knows about facade
@Component({
  selector: 'app-dashboard',
  changeDetection: ChangeDetectionStrategy.OnPush,
})
export class DashboardComponent {
  protected readonly facade = inject(AccountFacade);
  // No ApiService, no Store, no NotificationService directly
}
```

Code walkthrough — how this becomes the Facade Pattern:

1. **AccountFacade injects four dependencies** — `AccountApiService`, `TransactionApiService`, `NotificationService`, and `Store`. These four are the **complex subsystem**. The facade is the single point that knows all of them.
2. **readonly accounts and readonly isLoading** — Signals derived from the `NgRx` store. The component reads these directly without ever importing `Store` or knowing what selector drives them.
3. **loadDashboard(userId)** — One method call dispatches an `NgRx` action. The component does not import `AccountActions`, does not know what action type is dispatched, does not know the store exists.
4. **transferFunds()** — Orchestrates three internal steps: makes the API call, shows a success notification, and dispatches a reload action. It also owns the error handling. The component calls one method and receives a `Promise` — the entire workflow is hidden.
5. **DashboardComponent** — Injects only `AccountFacade`. The component's imports list has one item. No `Store`, no `AccountApiService`, no `NotificationService`. If any of the underlying services change their API, `DashboardComponent` is untouched.

Pattern conclusion: Instead of the component knowing about four services and their separate APIs, it knows about one — the facade. The facade is the **single point of contact**. This is Facade: a simplified interface over a complex subsystem. Refactoring an underlying service changes one file (the facade), not every component that uses it.

Real-world use: Banking dashboard facade, checkout facade in e-commerce, user profile facade.

Adapter Pattern

What it is: Converts the interface of one class into another interface that clients expect. Allows incompatible interfaces to work together.

How it helps: When integrating a third-party API that returns a different data shape, the adapter normalises it — components never see the raw API format.

Angular 21 implementation:

```
// legacy-account.adapter.ts
export interface LegacyAccountDto {
  acct_no: string;
  acct_type: string;
  bal: number;
  ccy: string;
}

export interface Account {
  id: string;
  type: 'current' | 'savings' | 'isa';
  balance: number;
  currency: string;
}

export function adaptLegacyAccount(dto: LegacyAccountDto): Account {
  return {
    id: dto.acct_no,
    type: mapAccountType(dto.acct_type),
    balance: dto.bal,
    currency: dto.ccy,
  };
}

function mapAccountType(raw: string): Account['type'] {
  const map: Record<string, Account['type']> = {
    CA: 'current',
    SA: 'savings',
    ISA: 'isa',
  };
  return map[raw] ?? 'current';
}

// In the API service
@Injectable({ providedIn: 'root' })
export class AccountApiService {
  getAccounts(): Observable<Account[]> {
    return this.http
      .get<LegacyAccountDto[]>('/legacy/api/accounts')
      .pipe(map((dtos) => dtos.map(adaptLegacyAccount)));
  }
}
```

Code walkthrough — how this becomes the Adapter Pattern:

1. **LegacyAccountDto** — The **incompatible interface** from the legacy API. Fields have cryptic names (`acct_no` , `bal` , `ccy`) and legacy type codes (`'CA'` , `'SA'`). This is what the third-party or legacy system sends — you cannot change it.
2. **Account** — The **target interface** your Angular app expects. Clean, descriptive names. This is what all components, services, and state management code works with.
3. **adaptLegacyAccount(dto)** — **This is the Adapter**. It takes the incompatible type and returns the target type. Each field mapping is explicit: `dto.acct_no` → `id` , `dto.bal` → `balance` , `dto.ccy` → `currency` . No magic, no mutation — pure transformation.
4. **mapAccountType()** — A helper that translates legacy string codes to a typed union. This is part of the adapter's responsibility: not just renaming fields, but translating domain concepts.
5. **AccountApiService.getAccounts()** — The HTTP response type is `LegacyAccountDto[]` . The return type is `Observable<Account[]>` . The `.pipe(map(dtos => dtos.map(adaptLegacyAccount)))` is the adapter in action — the transformation happens once, at the boundary. The rest of the app never imports or touches `LegacyAccountDto` .

Pattern conclusion: The legacy API and your application speak different "languages". The Adapter is the translator. All legacy complexity is contained in one file. Every component, every service, every test in your application works with the clean `Account` interface. When the legacy API changes its field names, you update one file — the adapter.

Real-world use: AngularJS → Angular 21 migration (adapting old REST endpoints), integrating third-party payment SDKs, normalising multi-bank API responses.

Decorator Pattern

What it is: Adds behaviour to an object dynamically without modifying its class. Wraps the original object.

How it helps: Cross-cutting concerns (logging, caching, retry logic) are added at the service level without polluting business logic.

Angular 21 implementation:

```
// cached-account-api.service.ts - Decorator over AccountApiService
@Injectable()
export class CachedAccountApiService extends AccountApiService {
  private readonly cache = new Map<string, { data: Account[]; timestamp: number }>();
  private readonly TTL = 60_000; // 1 minute

  override getAccounts(): Observable<Account[]> {
    const cached = this.cache.get('accounts');
    if (cached && Date.now() - cached.timestamp < this.TTL) {
      return of(cached.data);
    }
    return super
      .getAccounts()
      .pipe(tap((data) => this.cache.set('accounts', { data, timestamp: Date.now() })));
  }
}

// app.config.ts - swap in the decorator
export const appConfig: ApplicationConfig = {
  providers: [{ provide: AccountApiService, useClass: CachedAccountApiService }],
};
```

Code walkthrough — how this becomes the Decorator Pattern:

1. **extends AccountApiService** — The Decorator inherits from the class it wraps. This is the structural key: `CachedAccountApiService` IS an `AccountApiService` (same interface, same method signatures). Consumers do not know they are talking to a wrapper.
2. **private readonly cache = new Map<...>()** — Internal state the original `AccountApiService` does not have. This is the **added behaviour** — the cache layer is entirely new, bolted on without touching the original class.
3. **override getAccounts()** — Intercepts the call before it reaches the real implementation. Angular's `override` keyword makes this explicit and catches typos at compile time.
4. **Cache hit check** — Reads the cache. If data exists and is less than 60 seconds old, returns `of(cached.data)` immediately. No HTTP request is made. The original service is bypassed entirely.
5. **super.getAccounts()** — Cache miss: delegates to the original service. The real service does not know it is being wrapped. It just runs normally.
6. **.pipe(tap(...))** — After the response arrives, stores it in the cache with a timestamp. Next call within 60 seconds will hit the cache.
7. **{ provide: AccountApiService, useClass: CachedAccountApiService }** — Angular's DI substitutes `CachedAccountApiService` everywhere `AccountApiService` is injected. **Zero changes in consuming code** — they still inject `AccountApiService` and call `getAccounts()`.

Pattern conclusion: The cache behaviour is added without modifying `AccountApiService`. The original class is unchanged, untouched, still testable on its own. Consumers are unaware of the wrapper — they call the same interface. This is Decorator: new behaviour added by wrapping, not by modifying.

Real-world use: Caching HTTP responses, adding analytics tracking to service calls, wrapping API services with retry/circuit-breaker logic.

Proxy Pattern

What it is: Provides a surrogate or placeholder that controls access to another object. The proxy implements the same interface.

How it helps: Lazy loading, access control, and HTTP interceptors all follow the Proxy pattern.

Angular 21 implementation — HTTP Interceptor as Proxy:

```
// auth.interceptor.ts
export const authInterceptor: HttpInterceptorFn = (req, next) => {
  const authService = inject(AuthService);
  const token = authService.token();

  if (!token) return next(req);

  const authorisedReq = req.clone({
    headers: req.headers.set('Authorization', `Bearer ${token}`),
  });

  return next(authorisedReq).pipe(
    catchError((error) => {
      if (error.status === 401) {
        authService.logout();
      }
      return throwError(() => error);
    })
  );
};
```

Code walkthrough — how this becomes the Proxy Pattern:

1. **HttpInterceptorFn** — Angular's functional interceptor sits in the HTTP pipeline, **between the component and the server**. Every outbound request passes through it — this is the Proxy position.
2. **inject(AuthService)** and **authService.token()** — The proxy reads the token from the singleton service. It handles this cross-cutting concern so that components and API services never have to.
3. **if (!token) return next(req)** — The proxy is transparent when there is nothing to do. Unauthenticated requests pass through unchanged. The component is not aware this check happened.
4. **req.clone({ headers: ... })** — HTTP requests are immutable. `clone()` creates a modified copy with the `Authorization` header injected. The component that originally called `this.http.get(...)` never modified the request — the proxy did it.
5. **catchError(error => ...)** — On a 401 Unauthorized response, the proxy calls `authService.logout()` and re-throws. The component that triggered the request does not need any 401-handling code — the proxy owns that responsibility.

Pattern conclusion: The component calls `this.http.get('/api/accounts')` as normal. It does not know a proxy exists. The interceptor — the Proxy — controls access to the HTTP resource: adding credentials on the way out and handling auth failures on the way back. This is exactly the Proxy pattern: a surrogate that controls and enriches access to another object.

Real-world use: Auth token injection, request/response logging, rate limiting, offline queue (service worker proxy).

2.3 Behavioural Patterns

Behavioural patterns deal with **communication and responsibility between objects**.

Observer Pattern

What it is: Defines a one-to-many dependency — when one object changes state, all dependents are notified automatically.

How it helps: Decouples publishers from subscribers. Components react to state changes without polling.

Angular 21 — Signals ARE the Observer pattern:

```
// notifications.service.ts
@Injectable({ providedIn: 'root' })
export class NotificationService {
  private readonly _messages = signal<Notification[]>([]);
  readonly messages = this._messages.asReadonly();
  readonly unreadCount = computed(() => this._messages().filter(m => !m.read).length);

  push(message: Notification): void {
    this._messages.update((msgs) => [...msgs, message]);
  }

  markRead(id: string): void {
    this._messages.update((msgs) => msgs.map(m => (m.id === id ? { ...m, read: true } : m)));
  }
}

// notification-bell.component.ts — Observer
@Component({ changeDetection: ChangeDetectionStrategy.OnPush })
export class NotificationBellComponent {
  protected readonly count = inject(NotificationService).unreadCount;
  // count() automatically re-renders when unreadCount changes — no subscription needed
}
```

RxJS Observer pattern (for async streams):

```
// When you need streams (WebSocket, HTTP polling)
readonly transactions$ = this.webSocketService.connect('/ws/transactions').pipe(
  filter(event => event.type === 'TRANSACTION'),
  map(event => event.payload as Transaction),
  shareReplay(1)
);
```

Code walkthrough — how this becomes the Observer Pattern:

1. `private readonly _messages = signal<Notification[]>([])` — The **Subject** (the Observable source). It holds the canonical list of notifications. `private` means only methods inside this service can change it.
2. `readonly messages = this._messages.asReadonly()` — A read-only **Observable view** of the list. Observers can read but cannot push new values directly.
3. `readonly unreadCount = computed(...)` — A **derived Observable**. It automatically recalculates whenever `_messages` changes. Any component that reads `unreadCount()` is an implicit subscriber to notification changes.
4. `push(message)` — Calls `update()` on the signal. This is the **notify** step. The signal graph propagates the change — every computed value and template binding that depends on `_messages` is re-evaluated automatically.
5. `NotificationBellComponent.count` — Reads `inject(NotificationService).unreadCount`. There is no `subscribe()` call. When `push()` is called anywhere in the app, Angular's change detection picks up the signal change and re-renders this component's template. No manual subscription, no memory leak from forgetting to unsubscribe.

Pattern conclusion: `_messages` is the publisher. `NotificationBellComponent` (and any other component reading the signal) is the subscriber. The signal graph IS the Observer pattern: when the source changes, all dependents are notified automatically. Angular Signals eliminate the boilerplate (subscribe/unsubscribe) while preserving the pattern's intent.

Real-world use: Live transaction feeds, notification centres, real-time dashboard updates.

Strategy Pattern

What it is: Defines a family of algorithms, encapsulates each one, and makes them interchangeable. The algorithm varies independently from the clients that use it.

How it helps: Swap sorting, validation, or formatting logic at runtime without `if/switch` chains.

Angular 21 implementation:

```
// transaction-sort.strategy.ts
export type SortStrategy = (a: Transaction, b: Transaction) => number;

export const sortByDateDesc: SortStrategy = (a, b) =>
  new Date(b.date).getTime() - new Date(a.date).getTime();

export const sortByAmountDesc: SortStrategy = (a, b) => b.amount - a.amount;

export const sortByReference: SortStrategy = (a, b) => a.reference.localeCompare(b.reference);

// transaction-list.component.ts
@Component({ changeDetection: ChangeDetectionStrategy.OnPush })
export class TransactionListComponent {
  private readonly transactions = inject(TransactionService).transactions;
  protected readonly sortKey = signal<'date' | 'amount' | 'reference'>('date');

  private readonly strategies: Record<string, SortStrategy> = {
    date: sortByDateDesc,
    amount: sortByAmountDesc,
    reference: sortByReference,
  };

  protected readonly sorted = computed(() =>
    [...this.transactions()].sort(this.strategies[this.sortKey()]),
  );
}
```

Code walkthrough — how this becomes the Strategy Pattern:

1. **SortStrategy type** — A function signature that every strategy must satisfy: `(a: Transaction, b: Transaction) => number`. This is the **strategy interface** — the contract all algorithms share.
2. **sortByDateDesc, sortByAmountDesc, sortByReference** — Three **concrete strategies**, each a pure function. They are interchangeable because they all satisfy `SortStrategy`. None of them know about the component or each other.
3. **sortKey = signal<'date' | 'amount' | 'reference'>('date')** — The signal that determines which strategy is **active at runtime**. Changing this signal swaps the algorithm. No service restart, no component rebuild.
4. **strategies: Record<string, SortStrategy>** — A lookup map associating each key to its strategy function. Adding a new sort order is adding one entry to this map.
5. **sorted = computed(() => [...this.transactions()].sort(this.strategies[this.sortKey()])))** — The computed signal reads both `sortKey` and `transactions`. When either changes, it recomputes. It delegates to whichever strategy is currently selected — the `computed` itself does not care which algorithm runs.

Pattern conclusion: The sorting algorithm is a **variable** — a first-class object stored in a map and selected at runtime. There is no `if (sortKey === 'date') { ... } else if (sortKey === 'amount') { ... }` chain. Each algorithm is isolated, independently testable, and interchangeable. Adding a new sort option does not touch existing code — this is the Open/Closed Principle, enabled by Strategy.

Real-world use: Sorting/filtering strategies, payment validation strategies (UK sort code vs IBAN), export format strategies (CSV/PDF/Excel).

Command Pattern

What it is: Encapsulates a request as an object, allowing parameterisation, queuing, logging, and undo/redo.

How it helps: Undo/redo stacks, action queues, and audit logs all rely on commands being first-class objects.

Angular 21 implementation:

```

// command.interface.ts
export interface Command {
  execute(): void;
  undo(): void;
  description: string;
}

// transfer-funds.command.ts
export class TransferFundsCommand implements Command {
  readonly description: string;

  constructor(
    private readonly api: AccountApiService,
    private readonly store: Store,
    private readonly from: string,
    private readonly to: string,
    private readonly amount: number,
  ) {
    this.description = `Transfer £${amount} from ${from} to ${to}`;
  }

  execute(): void {
    this.api
      .transfer({ from: this.from, to: this.to, amount: this.amount })
      .pipe(
        tap(() => this.store.dispatch(AccountActions.reloadAccount({ id: this.from })))
      )
      .subscribe();
  }

  undo(): void {
    // Reverse transfer
    this.api.transfer({ from: this.to, to: this.from, amount: this.amount }).subscribe();
  }
}

// command-history.service.ts
@Injectable({ providedIn: 'root' })
export class CommandHistoryService {
  private readonly _history = signal<Command[]>([]);
  readonly history = this._history.asReadonly();

  execute(command: Command): void {
    command.execute();
    this._history.update((h) => [...h, command]);
  }

  undo(): void {
    const last = this._history()[this._history().length - 1];
    if (last) {
      last.undo();
      this._history.update((h) => h.slice(0, -1));
    }
  }
}

```

Code walkthrough — how this becomes the Command Pattern:

1. **Command interface** — Defines the contract: every command must have `execute()`, `undo()`, and a `description`. This is the **Command abstraction** — the history service only knows this interface, never the concrete command type.
2. **TransferFundsCommand constructor** — Captures all data needed at creation time: `api`, `store`, `from`, `to`, `amount`. The command is a **self-contained operation object** — everything required to execute (or undo) the operation is stored inside it.
3. **description** — A human-readable label. Useful for audit logs and undo history UI ("Transfer £500 from 12345678 to 87654321").
4. **execute()** — Performs the transfer via the API, then refreshes the store. This is the **forward operation**.
5. **undo()** — Performs a reverse transfer (same amount, from and to swapped). This is the **compensating operation** — what makes this pattern powerful for financial UIs.
6. **CommandHistoryService._history = signal<Command[]>([])** — The **command stack**. It holds all executed commands in order.
7. **execute(command)** — Runs the command and **pushes it onto the history stack**. The service does not know what the command does — it just calls `command.execute()` through the interface.
8. **undo()** — Pops the last command and calls `command.undo()`. Again, the service does not know the implementation — it only uses the `Command` interface.

Pattern conclusion: The transfer operation is encapsulated as an object with a known lifecycle: create → execute → optionally undo. The `CommandHistoryService` is the **invoker** — it manages the queue and calls the interface. The history service can undo any command without knowing anything about banking logic. New commands (e.g. `CreatePayeeCommand`) are added by implementing `Command` — no history service code changes.

Real-world use: Transaction undo in banking UIs, form wizard step history, admin bulk-action queues.

Mediator Pattern

What it is: Defines an object that encapsulates how a set of objects interact. Components communicate through the mediator rather than directly with each other.

How it helps: Reduces tight coupling between sibling components. The mediator (often a service or store) coordinates communication.

Angular 21 implementation:

```
// dashboard-mediator.service.ts
@Injectable({ providedIn: 'root' })
export class DashboardMediatorService {
  // Signals act as the shared communication channel
  private readonly _selectedAccount = signal<string | null>(null);
  readonly selectedAccount = this._selectedAccount.asReadOnly();

  private readonly _dateRange = signal<DateRange>({ from: startOfMonth(), to: new Date() });
  readonly dateRange = this._dateRange.asReadOnly();

  selectAccount(id: string): void {
    this._selectedAccount.set(id);
  }

  setDateRange(range: DateRange): void {
    this._dateRange.set(range);
  }
}

// account-list.component.ts — publishes to mediator
@Component({ changeDetection: ChangeDetectionStrategy.OnPush })
export class AccountListComponent {
  private readonly mediator = inject(DashboardMediatorService);
  onSelect(id: string): void {
    this.mediator.selectAccount(id);
  }
}

// transaction-panel.component.ts — subscribes from mediator
@Component({ changeDetection: ChangeDetectionStrategy.OnPush })
export class TransactionPanelComponent {
  protected readonly accountId = inject(DashboardMediatorService).selectedAccount;
  // Automatically reacts when selectedAccount signal changes
}
```

Code walkthrough — how this becomes the Mediator Pattern:

1. `DashboardMediatorService._selectedAccount` — A private writable signal. This is the **shared communication channel** — the Mediator's state. No component holds this state; it lives in the Mediator.
2. `readonly selectedAccount = this._selectedAccount.asReadonly()` — Public read-only access. Components can subscribe to changes but cannot write directly.
3. `selectAccount(id)` — A public write method. Any component can call this to announce "the user selected this account". The Mediator does not know — or care — which component called it.
4. `AccountListComponent` — Injects the Mediator and calls `mediator.selectAccount(id)` on user click. It does **not** hold a reference to `TransactionPanelComponent`. It does not know which other components care about account selection.
5. `TransactionPanelComponent.accountId` — Reads `mediator.selectedAccount`. When `AccountListComponent` calls `selectAccount()`, this signal updates automatically, and `TransactionPanelComponent` re-renders. The two components have **never referenced each other**.

Pattern conclusion: Without a Mediator, `AccountListComponent` would hold a reference to `TransactionPanelComponent` and call methods directly — tight coupling. With the Mediator, both components only know the Mediator service. The Mediator reduces the connection count from $n \times n$ (components referencing each other) to $n \times 1$ (every component talks only to the Mediator). Adding a new panel that needs to react to account selection means injecting the Mediator — zero changes to existing components.

Real-world use: Dashboard coordination between account list, transaction panel, and chart. The mediator replaces a tangle of `@Output()` event chains.

State Pattern

What it is: Allows an object to alter its behaviour when its internal state changes. The object appears to change its class.

How it helps: Complex UI workflows (loan application, onboarding wizard) have states that change available actions. The pattern makes state transitions explicit.

Angular 21 — Signal-based State Machine:

```
// loan-application-state.ts
export type LoanState =
  | 'idle'
  | 'personal-details'
  | 'income-check'
  | 'credit-check'
  | 'decision'
  | 'approved'
  | 'declined';

const TRANSITIONS: Record<LoanState, LoanState[]> = {
  idle: ['personal-details'],
  'personal-details': ['income-check'],
  'income-check': ['credit-check'],
  'credit-check': ['decision'],
  decision: ['approved', 'declined'],
  approved: [],
  declined: ['personal-details'],
};

@Injectable({ providedIn: 'root' })
export class LoanApplicationService {
  private readonly _state = signal<LoanState>('idle');
  readonly state = this._state.asReadonly();

  readonly canSubmit = computed(() => this._state() !== 'approved' && this._state() !== 'declined');

  transition(to: LoanState): void {
    const allowed = TRANSITIONS[this._state()];
    if (!allowed.includes(to)) {
      throw new Error(`Invalid transition: ${this._state()} → ${to}`);
    }
    this._state.set(to);
  }
}
```

Code walkthrough — how this becomes the State Pattern:

1. **LoanState union type** — Enumerates every possible state the application can be in. These are the **state nodes** in the machine. The type system prevents any other string from being used.
2. **TRANSITIONS map** — Defines which states can follow which. 'decision' can go to 'approved' or 'declined'. 'approved' has an empty array — there is no valid next state. Illegal transitions are structurally impossible.
3. **_state = signal<LoanState>('idle')** — A single signal holds the current state. The entire machine's memory is this one value.
4. **canSubmit = computed(...)** — A derived signal that changes behaviour based on state. In 'approved' or 'declined' the submit button should be disabled. The template reads `canSubmit()` — no `if` chains needed.
5. **transition(to)** — The **guard function**. It reads `TRANSITIONS[current state]` and throws if the requested transition is not in the allowed list. Only valid transitions call `_state.set(to)`. Invalid transitions are caught early with a descriptive error rather than silently corrupting state.

Pattern conclusion: The object's available actions change depending on `_state`. The `TRANSITIONS` map is a declarative replacement for complex `if/switch` logic. A developer reading the code immediately sees the full state graph. Adding a new state (e.g. 'expired') means adding one entry to `LoanState` and one row in `TRANSITIONS` — no existing transition logic changes.

Real-world use: Loan/mortgage application wizard, KYC onboarding flow, checkout state machine (cart → address → payment → confirmation).

Chain of Responsibility Pattern

What it is: Passes a request along a chain of handlers. Each handler decides to process it or pass it to the next handler.

How it helps: Middleware pipelines, Angular's HTTP interceptors, and form validators all follow this pattern.

Angular 21 — Custom Validator Chain:

```
// validator-chain.ts
export type ValidatorFn = (value: string) => string | null;

export function createValidatorChain(...validators: ValidatorFn[]): ValidatorFn {
  return (value: string): string | null => {
    for (const validator of validators) {
      const error = validator(value);
      if (error) return error; // First failure stops the chain
    }
    return null;
  };
}

const requiredValidator: ValidatorFn = (v) => (v.trim() ? null : 'This field is required');
const minLengthValidator =
  (min: number): ValidatorFn =>
  (v) =>
    v.length >= min ? null : `Minimum ${min} characters`;
const sortCodeValidator: ValidatorFn = (v) =>
  /^d{2}-d{2}-d{2}$/.test(v) ? null : 'Sort code must be in format 00-00-00';

// Usage in component with signal forms
const sortCodeValidate = createValidatorChain(
  requiredValidator,
  minLengthValidator(8),
  sortCodeValidator,
);
```


Code walkthrough — how this becomes the Chain of Responsibility Pattern:

1. **ValidatorFn type** — Every handler in the chain must match this signature: takes a string, returns a string error or `null`. This is the **handler interface** — the chain does not care about implementation, only the contract.
2. **createValidatorChain(...validators)** — The **chain builder**. It receives any number of handler functions and returns a new single function that runs them in sequence. The caller assembles the chain; the builder links the handlers.
3. **for (const validator of validators)** — Iterates through each handler in the order they were provided.
4. **if (error) return error** — **This is the chain-breaking behaviour**. The first handler that finds a problem returns its error immediately. The remaining handlers in the chain do not run. This matches the GoF intent: each handler either handles the request (returns an error) or passes it on (`null`).
5. **return null** — If all handlers pass, the chain concludes with no error — the value is valid.
6. **requiredValidator, minLengthValidator(8), sortCodeValidator** — Three concrete handlers, each responsible for exactly one validation rule. None of them knows about the others.
7. **createValidatorChain(required, minLength(8), sortCode)** — Assembles the chain. Order matters: `required` runs first (cheapest check), then length, then format. This mirrors how HTTP middleware is ordered: auth before parsing, parsing before business logic.

Pattern conclusion: Each validator is a single-responsibility handler. None knows about the others. The chain builder links them without any handler needing modification. Adding a new rule (e.g. a `notBlacklistedValidator`) is passing it as an extra argument to `createValidatorChain` — existing validators are unchanged. This is Chain of Responsibility: a request (the field value) passes through a sequence of handlers until one handles it (returns an error) or all pass.

Real-world use: Form field validation chains, Angular HTTP interceptor pipeline, middleware for auth → logging → error handling.

3. Architectural Patterns

Architectural patterns are **high-level strategies** for organising the entire application structure, team boundaries, and data flow.

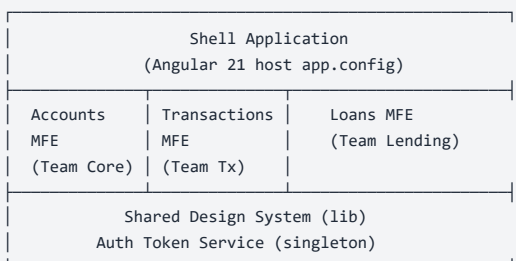
3.1 Micro-Frontend (MFE)

What it is: Splits a large frontend application into smaller, independently deployable units (micro-frontends), each owned by a separate team.

How it helps:

- Independent deployment — Team A ships without waiting for Team B
- Technology isolation — one MFE can be on Angular 21, another still on Angular 15 during migration
- Fault isolation — a crashed MFE widget does not crash the shell

Architecture diagram:



Angular 21 implementation with Module Federation:

```
// federation.config.js - Accounts MFE (remote)
const { withModuleFederationPlugin } = require('@angular-architects/module-federation/webpack');

module.exports = withModuleFederationPlugin({
  name: 'accounts',
  exposes: {
    './AccountsModule': './src/app/features/accounts/accounts.routes.ts',
  },
  shared: {
    '@angular/core': { singleton: true, strictVersion: true },
    '@angular/router': { singleton: true, strictVersion: true },
  },
});
```

```
// shell app.routes.ts - Shell loading remote
export const routes: Routes = [
  {
    path: 'accounts',
    loadChildren: () =>
      loadRemoteModule({
        type: 'module',
        remoteEntry: environment.accountsMfeUrl,
        exposedModule: './AccountsModule',
      }).then((m) => m.accountsRoutes),
  },
];
```

Communication between MFEs:

```
// shared-event-bus.service.ts - in shared lib
@Injectable({ providedIn: 'root' })
export class EventBusService {
  private readonly events = new Subject<AppEvent>();
  readonly events$ = this.events.asObservable();

  emit(event: AppEvent): void {
    this.events.next(event);
  }
}
```

Real-world use: Large banking portals (retail + business banking in same portal), insurance platforms with separate claim/policy/profile teams.

3.2 Atomic Design

What it is: Brad Frost's system for building design systems from the smallest parts (atoms) up to full pages. Five levels: Atoms → Molecules → Organisms → Templates → Pages.

Angular 21 folder structure:

```

src/
└─ lib/
  │   └─ atoms/
  │       │   └─ button/
  │       │   └─ input/
  │       │   └─ badge/
  │       │   └─ icon/
  │       └─ molecules/
  │           │   └─ form-field/      (label + input + error)
  │           │   └─ account-card/    (icon + balance + label)
  │           └─ search-bar/          (input + button)
  └─ organisms/
      │   └─ transaction-table/
      │   └─ account-summary/
      │   └─ navigation-header/
  └─ templates/
      │   └─ dashboard-layout/
      │   └─ form-page-layout/
  └─ pages/
      │   └─ dashboard-page/
      │   └─ transfer-page/

```

Atom example — Button:

```

// atoms/button/button.component.ts
@Component({
  selector: 'lib-button',
  standalone: true,
  changeDetection: ChangeDetectionStrategy.OnPush,
  host: { role: 'none' }, // host element is layout; inner <button> has role
  template: `
    <button
      [type]="type()"
      [disabled]="disabled()"
      [attr.aria-busy]="loading() ? 'true' : null"
      [class]="buttonClasses()"
    >
      @if (loading()) {
        <span aria-hidden="true" class="spinner" />
        <span class="sr-only">Loading...</span>
      } @else {
        <ng-content />
      }
    </button>
  `,
})
export class ButtonComponent {
  readonly type = input<'button' | 'submit' | 'reset'>('button');
  readonly variant = input<'primary' | 'secondary' | 'danger'>('primary');
  readonly disabled = input(false);
  readonly loading = input(false);

  protected readonly buttonClasses = computed(
    () => `btn btn--${this.variant()} ${this.loading() ? 'btn--loading' : ''}`,
  );
}

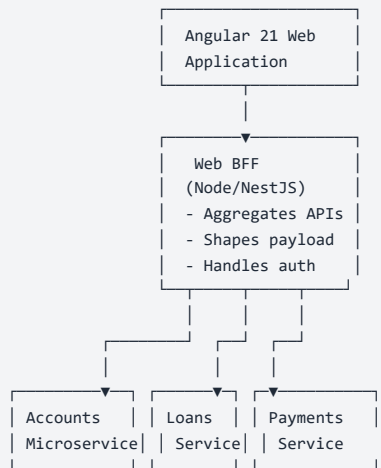
```

Real-world use: Enterprise design systems, component libraries shared across MFEs, brand-consistent UI across products.

3.3 Backend for Frontend (BFF)

What it is: A dedicated backend layer per frontend client (web, mobile, third-party). Each BFF aggregates and shapes data specifically for its client.

Architecture diagram:



Angular 21 — consuming BFF:

```
// account-bff.service.ts
@Injectable({ providedIn: 'root' })
export class AccountBffService {
  // BFF returns a pre-aggregated DashboardViewModel – no multiple round trips
  getDashboard(): Observable<DashboardViewModel> {
    return this.http.get<DashboardViewModel>('/bff/dashboard');
  }
}

// BFF response already has accounts + recent transactions + credit limit
// vs. 3 separate API calls the component would otherwise make
```

How it helps:

- Reduces round trips — one BFF call vs. 5 microservice calls
- Protects internal service topology from the browser
- Enables server-side token exchange (BFF holds backend credentials)
- SSR-friendly — Angular SSR can call BFF directly

Real-world use: Banking app (web sees accounts + transactions + alerts in one call), mobile BFF has a different payload optimised for small screens.

3.4 Saga Pattern

What it is: Manages long-running distributed transactions using a sequence of local transactions with compensating actions on failure. In Angular, NgRx Effects implement the Saga pattern.

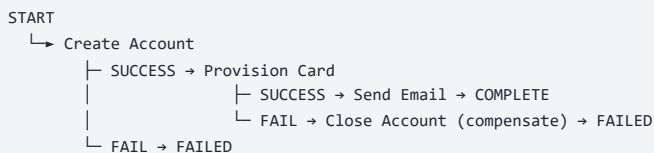
How it helps: When a business operation spans multiple API calls (create account → generate card → send welcome email), Sagas coordinate the steps and handle partial failures.

NgRx Effect as Saga:

```
// account-onboarding.effects.ts
@Injectable()
export class AccountOnboardingEffects {
  private readonly actions$ = inject(Actions);
  private readonly accountApi = inject(AccountApiService);
  private readonly cardApi = inject(CardApiService);
  private readonly emailApi = inject(EmailApiService);

  startOnboarding$ = createEffect(() =>
    this.actions$.pipe(
      ofType(OnboardingActions.start),
      switchMap(({ customerId }) =>
        this.accountApi.createAccount(customerId).pipe(
          // Step 2: provision card
          switchMap((account) =>
            this.cardApi.provisionCard(account.id).pipe(
              // Step 3: send welcome email
              switchMap((card) =>
                this.emailApi.sendWelcome({ customerId, accountId: account.id }).pipe(
                  map(() => OnboardingActions.complete({ account, card })),
                  // Compensating action if email fails
                  catchError(() => of(OnboardingActions.partialSuccess({ account, card }))),
                ),
              ),
            ),
            // Compensating action if card provisioning fails
            catchError((err) =>
              this.accountApi
                .closeAccount(account.id)
                .pipe(map(() => OnboardingActions.failed({ error: err.message }))),
            ),
          ),
        ),
        catchError((err) => of(OnboardingActions.failed({ error: err.message }))),
      ),
    ),
  );
}
```

Saga state flow:



Real-world use: Banking onboarding, loan disbursement (approve → disburse → notify), e-commerce checkout (reserve stock → charge card → dispatch).

3.5 CQRS & Event Sourcing

What it is:

- **CQRS (Command Query Responsibility Segregation):** Separates read operations (queries) from write operations (commands).
- **Event Sourcing:** Instead of storing current state, store the sequence of events that produced it.

Angular 21 — CQRS with NgRx:

```
// COMMANDS (write side) – dispatched actions
store.dispatch(AccountActions.debit({ accountId, amount }));
store.dispatch(AccountActions.credit({ accountId, amount }));

// QUERIES (read side) – selectors
const balance$ = store.select(selectAccountBalance(accountId));
const transactions$ = store.select(selectAccountTransactions(accountId));

// The reducer handles commands and produces new read-model state
const accountReducer = createReducer(
  initialState,
  on(AccountActions.debit, (state, { accountId, amount }) => ({
    ...state,
    accounts: state.accounts.map((acc) =>
      acc.id === accountId ? { ...acc, balance: acc.balance - amount } : acc,
    ),
  })),
);
```

Event Sourcing — transaction log:

```
// Every state change is an event
type AccountEvent =
| { type: 'ACCOUNT_OPENED'; accountId: string; openingBalance: number }
| { type: 'CREDITED'; accountId: string; amount: number; reference: string }
| { type: 'DEBITED'; accountId: string; amount: number; reference: string }
| { type: 'ACCOUNT_CLOSED'; accountId: string; reason: string };

// Replay events to reconstruct state
function replayAccount(events: AccountEvent[]): AccountState {
  return events.reduce(applyEvent, initialAccountState);
}
```

Real-world use: Financial ledgers (balance = replay of all debits/credits), audit trails, undo history.

3.6 Flux / NgRx (Redux)

What it is: Unidirectional data flow — Action → Reducer → State → View → Action.

Angular 21 NgRx architecture:

```
Component dispatches Action
↓
Effects (side effects: API calls)
↓
Reducer (pure function: state → new state)
↓
Store (single source of truth)
↓
Selector (memoised derived state)
↓
Component (via toSignal())
```

```
// Modern NgRx with signals
@Component({ changeDetection: ChangeDetectionStrategy.OnPush })
export class AccountsComponent {
  private readonly store = inject(Store);

  protected readonly accounts = toSignal(this.store.select(selectAllAccounts), {
    initialValue: [],
  });
  protected readonly isLoading = toSignal(this.store.select(selectAccountsLoading), {
    initialValue: false,
  });

  load(): void {
    this.store.dispatch(AccountActions.loadAccounts());
  }
}
```

When to use NgRx vs. Signals:

Scenario	Use
Local component state	<code>signal()</code>
Shared state between sibling components	Service with <code>signal()</code>
Complex async workflows (multiple effects)	NgRx
Cross-MFE shared state	NgRx or EventBus
Server state (caching, pagination)	Angular Query / TanStack Query

3.7 Smart / Presentational Components

What it is: Splits components into two roles:

- **Smart (Container):** Knows about the store/services. Fetches data. Dispatches actions.
- **Presentational (Dumb):** Receives data via `input()`. Emits events via `output()`. No service injection.

Angular 21 implementation:

```
// SMART - account-list-page.component.ts
@Component({
  selector: 'app-account-list-page',
  changeDetection: ChangeDetectionStrategy.OnPush,
  template: `
    <app-account-list
      [accounts]="accounts()"
      [isLoading]="isLoading()"
      (accountSelected)="onSelect($event)"
    />
  `,
})
export class AccountListPageComponent {
  private readonly store = inject(Store);
  protected readonly accounts = toSignal(this.store.select(selectAllAccounts), {
    initialValue: [],
  });
  protected readonly isLoading = toSignal(this.store.select(selectLoading), {
    initialValue: false,
  });

  protected onSelect(id: string): void {
    this.store.dispatch(AccountActions.select({ id }));
  }
}
```

```
// PRESENTATIONAL - account-list.component.ts
@Component({
  selector: 'app-account-list',
  changeDetection: ChangeDetectionStrategy.OnPush,
  // No inject() of any service
})
export class AccountListComponent {
  readonly accounts = input.required<Account[]>();
  readonly isLoading = input(false);
  readonly accountSelected = output<string>();
}
```

Real-world use: Every screen in a large app benefits from this split. Presentational components are 100% testable with just `ComponentFixture` and no mock services.

3.8 Feature-Sliced Design

What it is: An architectural methodology that organises code by **feature** and **layer**, not by file type. Each feature is a vertical slice containing its own components, services, state, and API.

Folder structure:

```
src/app/
├── features/
│   ├── accounts/
│   │   ├── components/      (Smart + presentational)
│   │   ├── services/        (AccountFacade, AccountApiService)
│   │   ├── state/           (NgRx actions, reducers, effects, selectors)
│   │   ├── models/          (Account interface, DTOs)
│   │   └── accounts.routes.ts
│   ├── transactions/
│   │   └── ...
│   ├── loans/
│   │   └── ...
│   └── shared/
│       ├── ui/              (Atomic design components)
│       ├── utils/           (Pure functions, helpers)
│       └── services/         (AuthService, NotificationService)
├── core/
│   ├── interceptors/
│   ├── guards/
│   └── app.config.ts
```

Rule: Features do not import from each other. They communicate through shared services or the store.

Real-world use: Teams map to features. The Accounts team owns `features/accounts/`, the Loans team owns `features/loans/`. No cross-team file conflicts.

4. Architectural Strategies

4.1 The 12-Factor App — Front-End Context

The 12-Factor App methodology (Heroku, 2011) defines principles for building scalable, maintainable software-as-a-service. Here is each factor mapped to Angular 21 front-end practice.

Factor	Principle	Angular 21 Application
I. Codebase	One codebase tracked in VCS; multiple deploys	Single git repo per app/MFE; Nx monorepo for multiple
II. Dependencies	Explicitly declare and isolate dependencies	<code>package.json</code> with exact versions; no CDN scripts in <code>index.html</code>
III. Config	Store config in the environment, not code	<code>environment.ts</code> / <code>environment.prod.ts</code> ; <code>APP_CONFIG</code> injection token
IV. Backing services	Treat APIs as attached resources	Abstract all API calls behind service classes; swap via DI
V. Build/Release/Run	Strictly separate build, release, and run stages	<code>ng build --configuration=production</code> ; Docker image per env
VI. Processes	Execute app as stateless processes	SSR stateless; no user state in module-level variables
VII. Port binding	Export services via port binding	Angular dev server on port 4200; SSR on 4000; MFEs on 4201+
VIII. Concurrency	Scale via process model	Multiple SSR instances behind load balancer
IX. Disposability	Fast startup and graceful shutdown	Lazy-loaded routes; <code>@defer</code> blocks; small initial bundle
X. Dev/prod parity	Keep dev, staging, and production as similar as possible	Docker Compose for local; same Angular build config
XI. Logs	Treat logs as event streams	<code>ErrorHandler</code> streams to logging service; no <code>console.log</code> in prod
XII. Admin processes	Run admin tasks as one-off processes	Seeding scripts separate from app; migration CLI tools

Angular 21 config injection token (Factor III):


```
// app-config.token.ts
export interface AppConfig {
  apiBaseUrl: string;
  featureFlags: Record<string, boolean>;
  environment: 'development' | 'staging' | 'production';
}

export const APP_CONFIG = new InjectionToken<AppConfig>('APP_CONFIG');

// app.config.ts
export const appConfig: ApplicationConfig = {
  providers: [
    {
      provide: APP_CONFIG,
      useValue: {
        apiBaseUrl: environment.apiBaseUrl,
        featureFlags: environment.featureFlags,
        environment: environment.name,
      },
    },
  ],
};
```

4.2 The 6-Hour Design Approach

The 6-Hour Design approach is a **time-boxed architectural decision framework** used when facing a new system or major feature. It forces architects to produce a shippable decision document in a single focused session.

The 6 hours, structured:

Hour	Activity	Output
Hour 1	Understand requirements — what problem are we solving?	Requirements doc, key constraints
Hour 2	Identify components — what are the major building blocks?	Component list, responsibility matrix
Hour 3	Draw the architecture — how do components connect?	C4 Context + Container diagrams
Hour 4	Identify risks — what can go wrong?	Risk register, ADR (Architecture Decision Record)
Hour 5	Define interfaces — how do components communicate?	API contracts, event schemas
Hour 6	Validate and decide — is this good enough to start?	ADR sign-off, backlog tickets

Architecture Decision Record (ADR) template:

```
# ADR-001: Adopt Micro-Frontend Architecture for Banking Portal

**Date:** 2026-05-21
**Status:** Accepted
**Context:** The banking portal has 4 teams working on overlapping features, causing deployment bottlenecks.
**Decision:** Adopt MFE with Module Federation. Each team owns one remote.
**Consequences:**

- ✅ Independent deployments per team
- ✅ Fault isolation between features
- ❌ Shared dependency version coordination required
- ❌ Initial setup overhead (~2 sprints)

**Alternatives considered:**

- Monolith with lazy modules — rejected (deployment coupling remains)
- iFrame-based MFE — rejected (accessibility and routing complexity)
```

5. Tools & Diagram Types

Diagramming Tools

Tool	Best For	Free?
draw.io / diagrams.net	All diagram types; exports SVG/PNG; Confluence plugin	Yes
Mermaid	Code-based diagrams in markdown; Git-friendly	Yes
Structurizr	C4 Model diagrams; code-as-architecture	Free tier
Lucidchart	Collaborative enterprise diagramming	Paid
PlantUML	UML from text; CI-friendly	Yes
Excalidraw	Whiteboard-style sketching	Yes
Figma / FigJam	Design system + architecture whiteboard	Paid

Diagram Types for Front-End Architecture

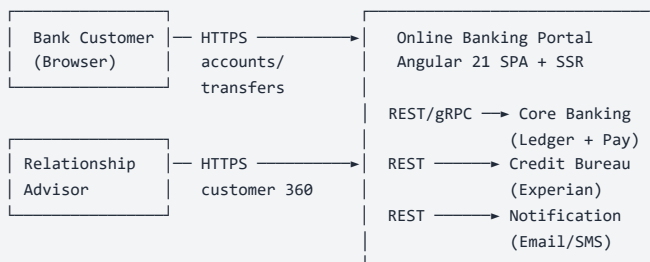
C4 Model (recommended for Angular apps)

Four levels of zoom:

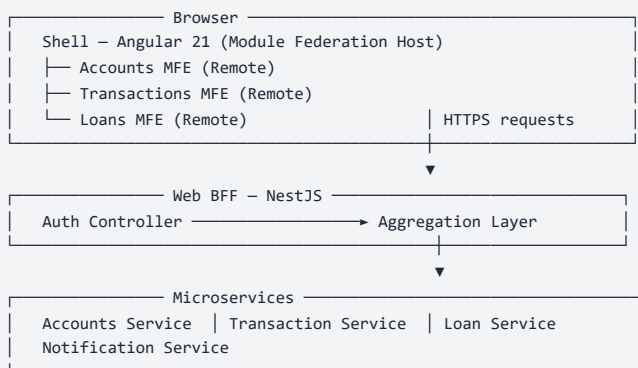
Level 1 – System Context	(who uses the system; what it connects to)
Level 2 – Container	(Angular app, BFF, microservices, databases)
Level 3 – Component	(NgRx store, feature modules, services inside the Angular app)
Level 4 – Code	(class diagrams for complex components)

Level 1 — System Context (Mermaid):

Note: C4Context syntax requires the Mermaid C4 plugin. The diagram below uses standard `graph LR` for maximum PDF/renderer compatibility. Semantically identical to a C4 Context diagram.



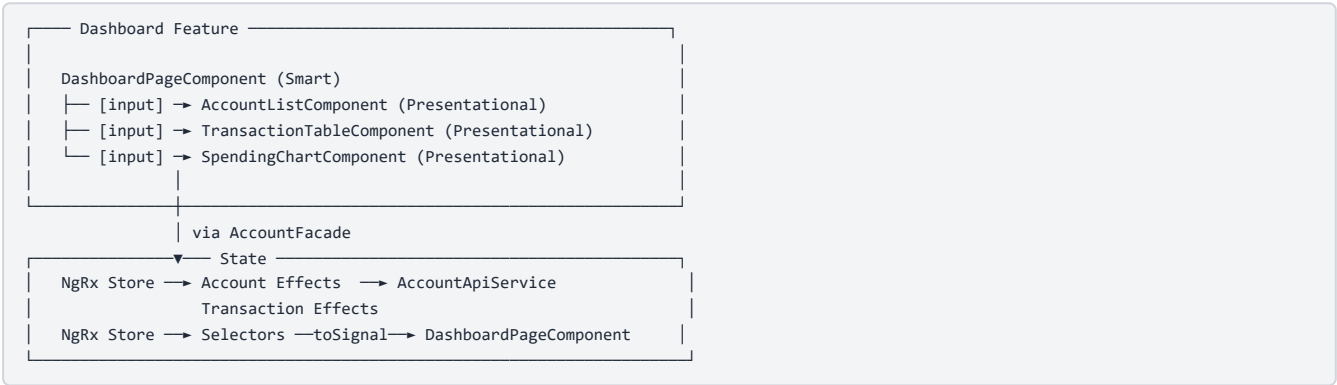
Level 2 — Container diagram (Mermaid):



Sequence Diagram — Fund Transfer Flow



Component Architecture Diagram



6. System Design Case Studies

6.1 Enterprise Banking Application

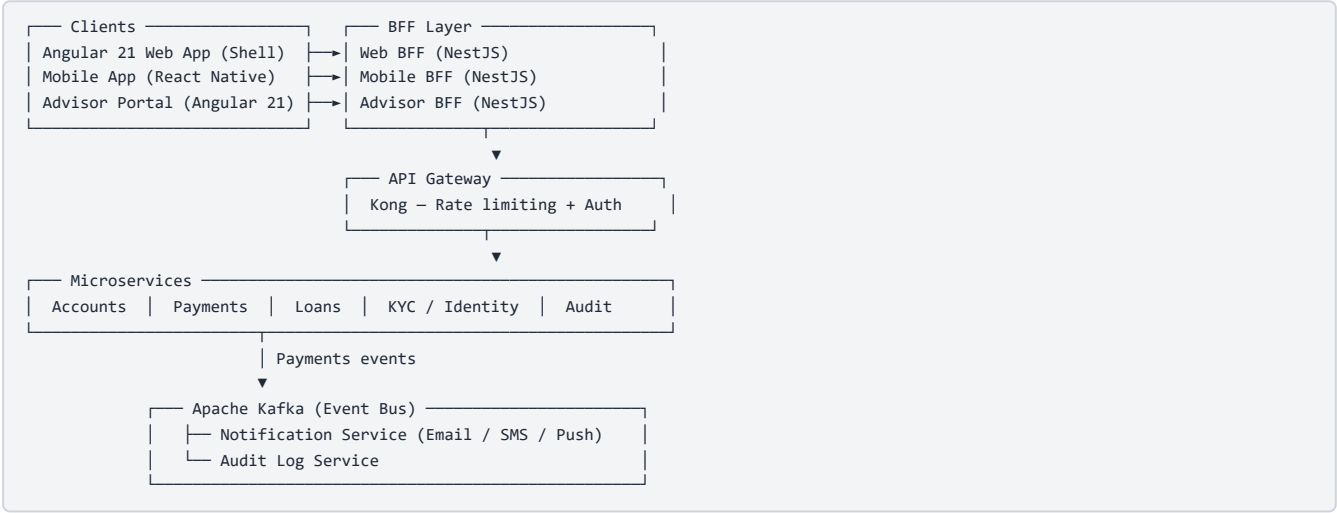
Requirements

Category	Requirement
Scale	2 million concurrent users at peak
Teams	8 product teams, 60+ engineers
Compliance	PSD2, GDPR, FCA regulations
Accessibility	WCAG 2.1 AA
Performance	LCP < 2.5s, CLS < 0.1, FID < 100ms
Deployment	Blue/green, zero-downtime

Architectural Decisions

Decision	Choice	Rationale
App structure	Micro-Frontend (Module Federation)	8 teams need independent deployments
State management	NgRx per MFE + Signal service for local state	Saga pattern for complex workflows; signals for UI state
API layer	BFF (NestJS) per client type	Aggregate microservices; handle auth token exchange
Design system	Atomic Design component library	Consistent brand across all MFEs
SSR	Angular Universal with hydration	SEO + initial load performance
Auth	OAuth2 / OIDC with PKCE	Regulatory requirement; BFF holds refresh tokens
Testing	Unit (Vitest) + Component (Angular CDK) + E2E (Cypress)	Full confidence at each layer

Full System Design (Mermaid)



Angular 21 Shell Implementation

```
// shell/app.config.ts
export const appConfig: ApplicationConfig = {
  providers: [
    provideRouter(routes, withPreloading(PreloadAllModules)),
    provideHttpClient(withInterceptors([authInterceptor, loggingInterceptor, errorInterceptor])),
    provideClientHydration(withEventReplay()),
    provideStore(),
    provideEffects(),
    provideStoreDevtools({ maxAge: 25 }),
    {
      provide: APP_CONFIG,
      useValue: {
        apiBaseUrl: environment.bffUrl,
        featureFlags: environment.featureFlags,
      },
    },
  ],
};
```

```
// shell/app.routes.ts
export const routes: Routes = [
  {
    path: '',
    canActivate: [authGuard],
    children: [
      {
        path: 'accounts',
        loadChildren: () =>
          loadRemoteModule({
            type: 'module',
            remoteEntry: `${environment.accountsMfeUrl}/remoteEntry.js`,
            exposedModule: './AccountsModule',
          }).then(m => m.accountsRoutes),
      },
      {
        path: 'payments',
        loadChildren: () =>
          loadRemoteModule({
            type: 'module',
            remoteEntry: `${environment.paymentsMfeUrl}/remoteEntry.js`,
            exposedModule: './PaymentsModule',
          }).then(m => m.paymentsRoutes),
      },
      {
        path: 'loans',
        @defer — loaded only when user has loan products
        loadChildren: () =>
          loadRemoteModule({
            type: 'module',
            remoteEntry: `${environment.loansMfeUrl}/remoteEntry.js`,
            exposedModule: './LoansModule',
          }).then(m => m.loansRoutes),
      },
    ],
  },
  { path: 'login', loadComponent: () => import('./login/login.component') },
  { path: '**', redirectTo: 'accounts' },
];
```

6.2 AngularJS → Angular 21.2 Migration

Context: A large enterprise banking app built in AngularJS 1.8 (2014–2019). 120+ controllers, 80+ services, 40+ directives. The goal is a zero-downtime, incremental migration to Angular 21.2 with SSR, Signals, and a modern design system — without a big-bang rewrite.

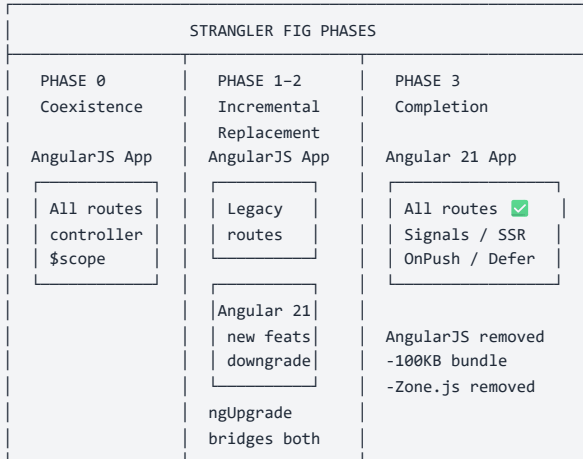
Strategy Comparison — Choose Your Path

Strategy	Risk	Speed	Team Disruption	Recommended For
1. Strangler Fig + ngUpgrade (Hybrid)	Medium	Slow–Medium	Low	Teams with AngularJS expertise, must maintain both
2. Nx Workspace — Shell + Dual Apps (MFE)	Low	Medium	Low	Large teams, feature-flag toggles, parallel delivery
3. AI-Assisted Full Rewrite (Copilot-Led)	Medium	Fast	Medium	Modern teams with good test coverage, tight deadlines
4. Route-by-Route Strangler (No ngUpgrade)	Low	Slow	Very Low	Small teams, low budget, risk-averse organisations
5. Big-Bang Rewrite	High	Fast on paper	High	✗ Not recommended — rarely succeeds at scale

Recommended approach for an enterprise banking app: Strategy 2 (Nx Shell + MFE) with Strategy 3 (AI-Assisted) powering the actual code conversion. Use the shell to safely toggle between old and new, and use Copilot to accelerate the per-feature rewrites.

Strategy 1 — Strangler Fig with ngUpgrade (Hybrid Bootstrap)

The Strangler Fig pattern grows the new Angular 21 app around the AngularJS shell — feature by feature, screen by screen — until the old system can be removed.



Phase 0 — Hybrid Bootstrap:

```
// main.ts - hybrid app bootstrap
import { setAngularJSGlobal } from '@angular/upgrade/static';
import * as angular from 'angular';
import { platformBrowserDynamic } from '@angular/platform-browser-dynamic';
import { AppModule } from './app/app.module';

setAngularJSGlobal/angular;
platformBrowserDynamic()
  .bootstrapModule(AppModule)
  .catch((err) => console.error(err));
```

Phase 1 — Downgrade Angular 21 component into AngularJS templates:

```
// New Angular 21 component
@Component({
  selector: 'app-account-card',
  standalone: true,
  changeDetection: ChangeDetectionStrategy.OnPush,
  template: `<div class="account-card">{{ account().name }}</div>`,
})
export class AccountCardComponent {
  readonly account = input.required<Account>();
}

// Register as an AngularJS directive via downgrade
import { downgradeComponent } from '@angular/upgrade/static';
angular
  .module('legacyApp')
  .directive('appAccountCard', downgradeComponent({ component: AccountCardComponent }));
```

```
<!-- AngularJS template now uses the Angular 21 component -->
<app-account-card [account]="ctrl.account"></app-account-card>
```

Phase 2 — Upgrade AngularJS service for use in Angular 21:

```
// Lift the legacy auth service into Angular's DI
export function legacyAuthFactory(i: any) {
  return i.get('LegacyAuthService');
}

export const LegacyAuthProvider = {
  provide: 'LegacyAuthService',
  useFactory: legacyAuthFactory,
  deps: ['$injector'],
};

// Angular 21 wrapper - forwards calls, adds type safety
@Injectable({ providedIn: 'root' })
export class AuthBridgeService {
  private legacy = inject('LegacyAuthService' as any);
  getToken(): string {
    return this.legacy.getToken();
  }
}
```

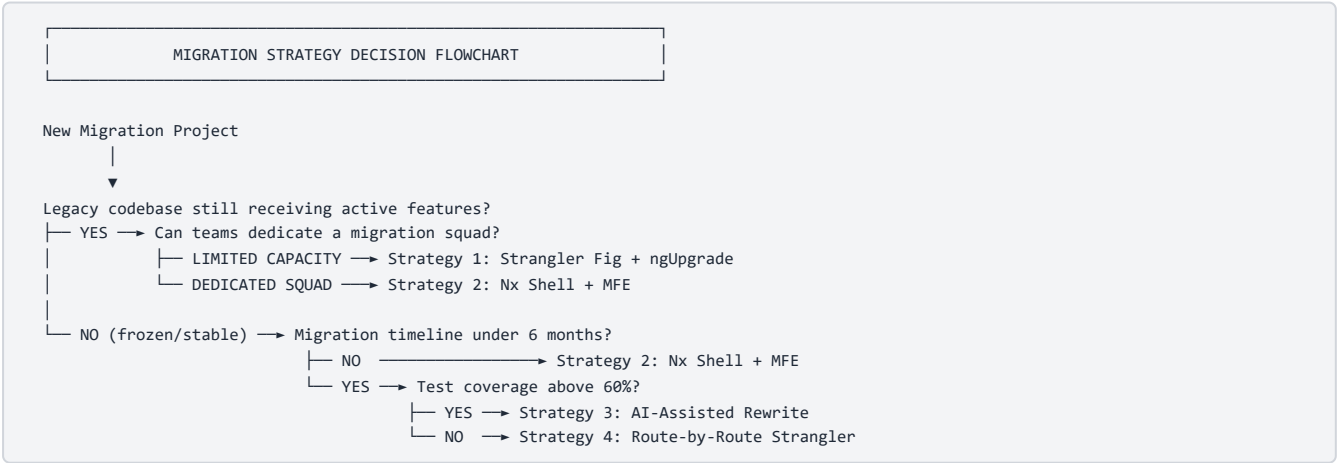
When to stop using ngUpgrade: Once all routes are migrated to Angular 21, remove `@angular/upgrade`, hybrid bootstrap, and AngularJS entirely. The full AngularJS bundle (~100 KB gzipped) disappears from the client.

Strangler Fig — When to Use This Pattern

Choose Strategy 1 when at least two of the following conditions are true:

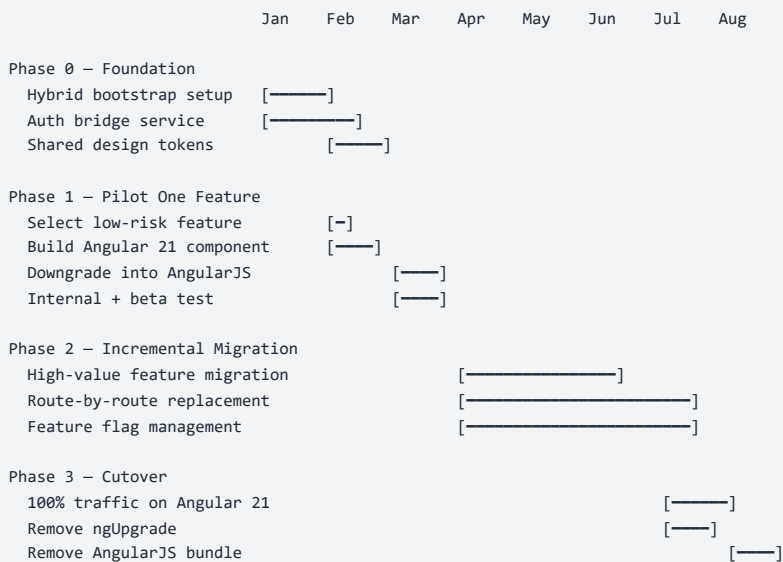
Condition	Why It Points to Strangler Fig
Legacy app still receives active feature development	You cannot freeze AngularJS; both codebases must coexist for 12+ months
Team has deep AngularJS expertise	Reskilling takes time — ngUpgrade lets devs work in their comfort zone while learning Angular 21
No dedicated migration budget	Strangler Fig progresses one component per sprint with no separate migration sprint
Complex domain logic with limited test coverage	Incremental migration lets you validate each piece before moving on
Risk-averse governance (banking, regulated industries)	Each piece can be rolled back independently; no big-bang risk

Migration Strategy Decision Flowchart



Migration Phase Timeline

Strangler Fig Migration Phases – 12-Month Timeline



Auth Bridge — Sharing Session State Across Both Frameworks

The critical challenge in a hybrid app is authentication. Both the AngularJS and Angular 21 zones must recognise the same user session without duplicating session management logic.

```
// shared-auth/src/lib/auth-bridge.service.ts
// Lives in a shared lib consumed by BOTH frameworks via their respective DI systems
@Injectable({ providedIn: 'root' })
export class AuthBridgeService {
  private readonly TOKEN_KEY = 'banking_access_token';

  // Internal writable signal – single source of truth
  private readonly _token = signal<string | null>(localStorage.getItem(this.TOKEN_KEY));

  // Angular 21 components read this signal reactively
  readonly token = this._token.asReadonly();
  readonly isAuthenticated = computed(() => this._token() !== null);

  setToken(token: string): void {
    localStorage.setItem(this.TOKEN_KEY, token);
    this._token.set(token);
    // Notify AngularJS side via a cross-framework DOM event
    window.dispatchEvent(new CustomEvent('auth:token-updated', { detail: { token } }));
  }

  clearToken(): void {
    localStorage.removeItem(this.TOKEN_KEY);
    this._token.set(null);
    window.dispatchEvent(new CustomEvent('auth:logout'));
  }
}
```



```
// AngularJS side – receive auth events dispatched by Angular 21
angular.module('legacyApp').run([
  '$rootScope',
  function ($rootScope) {
    window.addEventListener('auth:token-updated', function (event) {
      $rootScope.$apply(function () {
        $rootScope.authToken = event.detail.token;
      });
    });

    window.addEventListener('auth:logout', function () {
      $rootScope.$apply(function () {
        $rootScope.$broadcast('user:logout');
      });
    });
  },
]);
```

Zone.js Conflict Management

When both frameworks coexist, Zone.js runs under AngularJS while Angular 21 can operate zoneless. Without care, every user interaction triggers both an AngularJS `$digest` cycle and an Angular change detection cycle — doubling CPU cost.

```
// hybrid-safe.component.ts – avoid triggering both CD cycles
@Component({ changeDetection: ChangeDetectionStrategy.OnPush })
export class HybridSafeComponent {
  private readonly ngZone = inject(NgZone);
  protected readonly result = signal<ProcessedData | null>(null);

  processLargeDataset(data: RawData[]): void {
    // Run computation outside Angular zone – no CD triggered during processing
    this.ngZone.runOutsideAngular(() => {
      const processed = this.heavyTransform(data);

      // Re-enter Angular zone only for the final signal update
      this.ngZone.run(() => {
        this.result.set(processed);
      });
    });
  }
}
```

Rollback Triggers — Agree Before Starting

Define these signals before starting migration. Align with product and QA teams.

Signal	Threshold	Action
Error rate increase on migrated route	> 0.5% above baseline	Roll back feature flag immediately
Lighthouse LCP regression	Increases > 500 ms	Investigate before proceeding
Accessibility regression	Any new WCAG AA failure	Block release; fix before proceeding
User drop-off rate on migrated route	> 5% increase in bounce rate	A/B test; consider UX review
Bundle size increase	> 20 KB after migration	Optimise lazy loading before next sprint
<code>\$scope</code> references plateau	No reduction for 2 sprints	Review migration approach with team

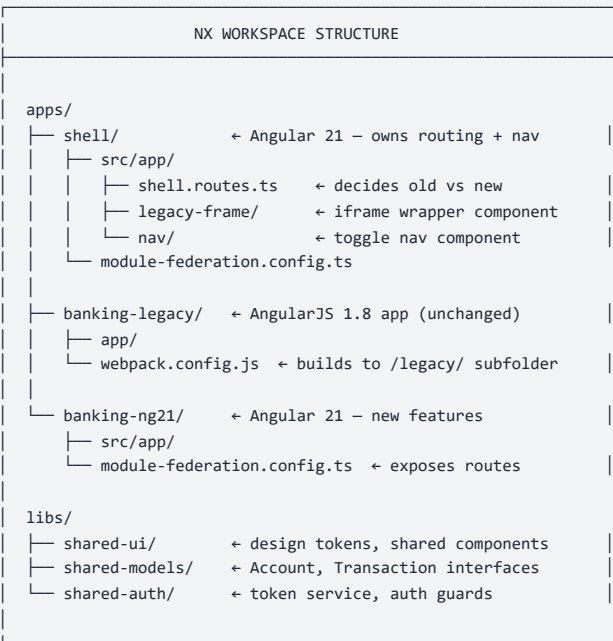
Migration Velocity Metrics — Track Per Sprint

Metric	Formula	Sprint Target
AngularJS code reduction	<code>deleted_AJS_lines / total_AJS_lines * 100</code>	-5% per sprint
Angular 21 component coverage	<code>ng21_components / total_components * 100</code>	+5% per sprint
Bundle size delta	<code>new_bundle_KB - previous_bundle_KB</code>	≤ 0 KB
Test coverage on migrated files	<code>vitest --coverage</code>	≥ 80%
Remaining <code>\$scope</code> references	<code>grep -r '\\$scope' src/ wc -l</code>	-10% per sprint

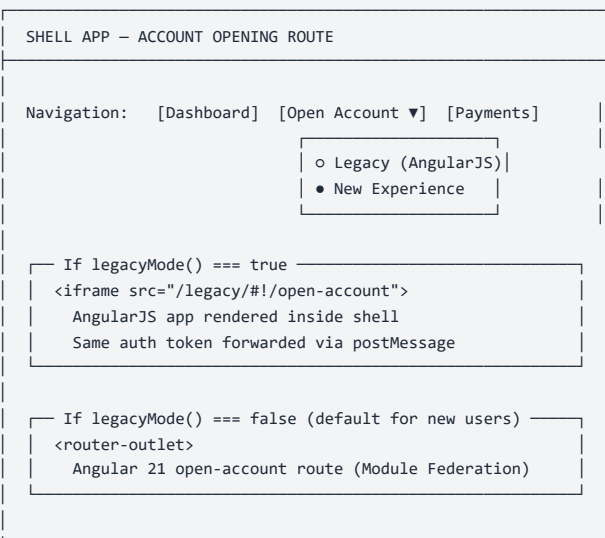
Architect's note: Strangler Fig is the safest strategy for organisations that cannot afford user-facing downtime or regression. Its weakness is duration — 12–18 months for a large app. Combine it with Strategy 3 (AI-Assisted Copilot conversion) to compress the per-feature timeline, and use Strategy 2 (Nx Shell + MFE) as the deployment harness so teams can ship independently while the migration progresses.

Strategy 2 — Nx Workspace: Shell App + Dual Apps (Recommended)

Run AngularJS and Angular 21 as **two separate applications** inside a single Nx monorepo. A thin Angular 21 **shell app** owns the navigation and decides which app renders each route — using Module Federation for the Angular 21 app and an iframe or web-component bridge for the legacy AngularJS app.



Navigation toggle in the shell — old vs new:



Shell routing — toggle signal drives which app loads:

```
// apps/shell/src/app/shell.routes.ts
export const SHELL_ROUTES: Routes = [
  {
    path: 'open-account',
    loadComponent: () =>
      import('./account-host/account-host.component').then((m) => m.AccountHostComponent),
  },
  {
    path: 'dashboard',
    loadChildren: () => import('banking-ng21/dashboard').then((m) => m.DASHBOARD_ROUTES),
  },
];
```

```
// apps/shell/src/app/account-host/account-host.component.ts
@Component({
  selector: 'app-account-host',
  standalone: true,
  changeDetection: ChangeDetectionStrategy.OnPush,
  imports: [NgIf, RouterOutlet, LegacyFrameComponent],
  template: `
    <div class="version-toggle" role="group" aria-label="Experience version">
      <button
        [class.active]="!legacyMode()"
        (click)="legacyMode.set(false)"
        [attr.aria-pressed]="!legacyMode()"
      >
        New Experience
      </button>
      <button
        [class.active]="legacyMode()"
        (click)="legacyMode.set(true)"
        [attr.aria-pressed]="legacyMode()"
      >
        Legacy
      </button>
    </div>

    @if (legacyMode()) {
      <!-- AngularJS app served at /legacy - bridged via iframe + postMessage -->
      <app-legacy-frame path="#!/open-account" />
    } @else {
      <!-- Angular 21 MFE route loaded via Module Federation -->
      <router-outlet />
    }
  `,
})
export class AccountHostComponent {
  /** Persisted in localStorage so user preference survives page reload */
  protected readonly legacyMode = signal(localStorage.getItem('preferLegacy') === 'true');

  constructor() {
    effect(() => {
      localStorage.setItem('preferLegacy', String(this.legacyMode()));
    });
  }
}
```

```
// apps/shell/src/app/legacy-frame/legacy-frame.component.ts
// Thin iframe wrapper – forwards auth token via postMessage
@Component({
  selector: 'app-legacy-frame',
  standalone: true,
  changeDetection: ChangeDetectionStrategy.OnPush,
  template: `
    <iframe
      [src]="legacyUrl()"
      title="Legacy account opening"
      class="legacy-frame"
      (load)="onLoad()"
    ></iframe>
  `,
})
export class LegacyFrameComponent {
  readonly path = input.required<string>();
  private auth = inject(AuthService);

  protected readonly legacyUrl = computed(() => `/legacy/index.html${this.path()}`);

  protected onLoad(): void {
    // Forward auth token to legacy app via postMessage (same-origin)
    const frame = document.querySelector('iframe') as HTMLIFrameElement;
    frame.contentWindow?.postMessage(
      { type: 'AUTH_TOKEN', token: this.auth.token() },
      window.location.origin,
    );
  }
}
```

Nx workspace setup commands:

```
# Create Nx workspace
npx create-nx-workspace@latest banking-workspace --preset=apps

# Add Angular 21 shell app
nx g @nx/angular:app shell --standalone --routing

# Add Angular 21 new banking app (MFE remote)
nx g @nx/angular:app banking-ng21 --standalone --routing

# Add Module Federation config to both
nx g @nx/angular:setup-mf shell --mfType=host --federationType=dynamic
nx g @nx/angular:setup-mf banking-ng21 --mfType=remote --port=4201

# Add legacy AngularJS app (existing – just register it in workspace)
# Add shared libs
nx g @nx/angular:lib shared-models
nx g @nx/angular:lib shared-auth
nx g @nx/angular:lib shared-ui
```

Feature flag strategy — roll out new experience progressively:

User segment → Feature flag → Which app

Internal staff	→ new (100%)	→ Angular 21
Beta users (opt-in)	→ new (100%)	→ Angular 21
New customers (post-go-live)	→ new	→ Angular 21
Existing customers	→ legacy	→ AngularJS
All users (final cutover)	→ new (100%)	→ Angular 21
AngularJS decommissioned	→	

Strategy 3 — AI-Assisted Migration with GitHub Copilot

AI tooling can convert AngularJS controllers, services, and templates to Angular 21 at a speed no manual effort matches. The key is **systematic prompting + automated verification** — not blind code generation.

AI-ASSISTED MIGRATION PIPELINE

1. AUDIT & CATALOGUE (AI reads, you confirm)

Copilot scans all .js files → produces migration-backlog.md:

Controller: AccountCtrl (\$scope, \$http)	Priority: H
Service: AuthService (\$cookies, \$http)	Priority: H
Directive: sortable-table (\$compile)	Priority: M
Filter: currencyFormat	Priority: L

2. PER-FILE CONVERSION (Copilot generates, dev reviews)

AngularJS Controller → Copilot prompt → Angular 21
\$scope variables → signal() / input()
\$http calls → inject(HttpClient) + Observable
\$watch expressions → computed() / effect()
ng-if / ng-repeat → @if / @for
\$routeParams → ActivatedRoute / routerInput()

3. AUTOMATED VERIFICATION

Copilot generates unit tests for each migrated file
ng build --prod (TypeScript strict mode catches drift)
Vitest coverage gate ≥ 80%

4. CODEMOD LAYER (automated, no AI needed)

ng update @angular/core → auto-migrates deprecated APIs
@angular/codemod → *ngIf → @if, *ngFor → @for
eslint --fix → lint rule auto-fixes

Copilot prompt template — converting a controller:

You are an Angular 21 expert. Convert the following AngularJS 1.8 controller to an Angular 21 standalone component. Apply these rules:

- Use signal() for all reactive state (replaces \$scope variables)
- Use inject() for all dependencies (no constructor injection)
- Use input() / output() for component I/O (no @Input/@Output decorators)
- Use computed() for derived state (replaces \$watch with return value)
- Use effect() for side effects (replaces \$watch with no return)
- Replace \$http with inject(HttpClient)
- Replace ng-if/ng-repeat in template with @if / @for blocks
- Use ChangeDetectionStrategy.OnPush
- Use Angular Signals-based forms, not ReactiveFormsModule
- Output a fully typed TypeScript file with strict null checks

AngularJS controller to migrate:

[PASTE CONTROLLER CODE HERE]

Copilot prompt — converting a service:

Convert this AngularJS service/factory to an Angular 21 injectable service.

Rules:

- @Injectable({ providedIn: 'root' }) – singleton
- Replace \$http with HttpClient
- Replace \$q promises with RxJS Observable or async/await
- Replace \$rootScope.\$broadcast with a signal or Subject
- Preserve all method names (existing consumers depend on them)
- Add full TypeScript types

AngularJS service:

[PASTE SERVICE CODE HERE]

Automated codemod pipeline (run before any manual migration):

```
# Step 1 – update Angular to latest (handles framework deprecations)
npx ng update @angular/core @angular/cli

# Step 2 – run official Angular migration schematic
npx ng update @angular/core --migrate-only --from=1 --to=21

# Step 3 – convert structural directives to control flow blocks
npx @angular/codemod migrate-control-flow ./src

# Step 4 – standalone migration (removes NgModule boilerplate)
npx ng generate @angular/core:standalone

# Step 5 – inject() migration (replaces constructor injection)
npx ng generate @angular/core:inject

# Step 6 – ESLint auto-fix
npx eslint ./src --fix --rule '{"@angular-eslint/no-input-rename": "error"}'
```

Conversion mapping — AngularJS to Angular 21:

AngularJS concept	Angular 21 equivalent	Notes
<code>\$scope.foo = bar</code>	<code>foo = signal(bar)</code>	Reactive, no zone needed
<code>\$watch('foo', fn)</code>	<code>effect(() => { fn(foo()) })</code>	Auto-tracks dependencies
<code>\$http.get(url)</code>	<code>inject(HttpClient).get(url)</code>	Returns <code>Observable</code>
<code>\$q.defer()</code>	<code>firstValueFrom(observable)</code>	Native <code>async/await</code>
<code>\$routeParams.id</code>	<code>inject(ActivatedRoute).snapshot.params['id']</code>	Or <code>routerInput()</code> signal
<code>\$rootScope.\$broadcast</code>	<code>inject(EventBusService) signal</code>	Replace with mediator pattern
<code>\$filter('currency')</code>	<code>CurrencyPipe</code> or <code>computed()</code>	Angular built-in pipe
<code>ng-if</code>	<code>@if</code> block	No directive import needed
<code>ng-repeat</code>	<code>@for</code> block	Track by built in
<code>ng-class</code>	<code>[class.name]="condition"</code>	Attribute binding
<code>\$compile</code> (dynamic)	<code>@defer</code> + lazy route	Avoid dynamic compilation
<code>angular.module</code>	<code>bootstrapApplication()</code>	Standalone bootstrap
Controller	Standalone component	<code>\$scope</code> → signals
Factory / Service	<code>@Injectable</code> service	Same singleton behaviour
Directive (structural)	<code>@if</code> / <code>@for</code> / <code>@switch</code>	Native control flow
Directive (attribute)	Angular Directive or Pipe	Depends on purpose

Strategy 4 — Route-by-Route Strangler (No ngUpgrade)

Redirect one Angular Router route at a time to the new Angular 21 app. The AngularJS app stays completely unchanged. No hybrid bootstrap, no ngUpgrade dependency.

ROUTE-BY-ROUTE STRANGLER

Reverse Proxy (nginx / CDN)

/dashboard	→ Angular 21 app	✓
/open-account	→ Angular 21 app	✓
/payments	→ Angular 21 app	✓
/statements	→ AngularJS app	(in progress)
/settings	→ AngularJS app	(pending)
/*	→ AngularJS app	(catch-all)

Shared session cookie / JWT – both apps read the same auth token from localStorage

No iframe, no ngUpgrade – clean separation

```
# nginx.conf – route-by-route strangler
location /dashboard {
    proxy_pass http://angular21-app:4200;
}
location /open-account {
    proxy_pass http://angular21-app:4200;
}
# All other routes → legacy AngularJS
location / {
    proxy_pass http://angularjs-app:3000;
}
```

Limitation: Users notice a full page reload crossing the boundary. Mitigate with a shared loading indicator injected into both apps' `<head>` .

Phase 3 — Angular 21.2 New Features to Adopt Post-Migration

Once the AngularJS shell is removed, immediately adopt these Angular 21 patterns to get the full performance benefit of migration:

```
// Signal-based form – replaces AngularJS ngModel + $watch
@Component({
  selector: 'app-transfer-form',
  standalone: true,
  changeDetection: ChangeDetectionStrategy.OnPush,
  template: `
    <form (ngSubmit)="submit()">
      <label for="from">From Account</label>
      <input
        id="from"
        [value]="fromAccount()"
        (input)="fromAccount.set($any($event.target).value)"
      />
      @if (errors().fromAccount) {
        <p role="alert">{{ errors().fromAccount }}</p>
      }

      <label for="amount">Amount</label>
      <input
        id="amount"
        type="number"
        [value]="amount()"
        (input)="amount.set(+$any($event.target).value)"
      />
      @if (errors().amount) {
        <p role="alert">{{ errors().amount }}</p>
      }

      <button type="submit" [disabled]="!isValid()">Transfer</button>
    </form>
  `,
})
export class TransferFormComponent {
  protected readonly fromAccount = signal('');
  protected readonly amount = signal<number | null>(null);

  protected readonly errors = computed(() => ({
    fromAccount: !this.fromAccount().trim() ? 'Please select an account' : null,
    amount: (this.amount() ?? 0) <= 0 ? 'Amount must be greater than zero' : null,
  }));

  protected readonly isValid = computed(() =>
    Object.values(this.errors()).every((e) => e === null),
  );

  protected submit(): void {
    if (!this.isValid()) return;
  }
}
```

```
// @defer – replaces AngularJS ng-if lazy pattern
// template
`
@defer (on viewport) {
  <app-spending-chart [data]="chartData()" />
} @placeholder {
  <div class="chart-skeleton" aria-busy="true">Loading chart...</div>
} @error {
  <p>Chart could not load.</p>
}
`;
```

6.3 MFE Deployment Topologies for Legacy Migration

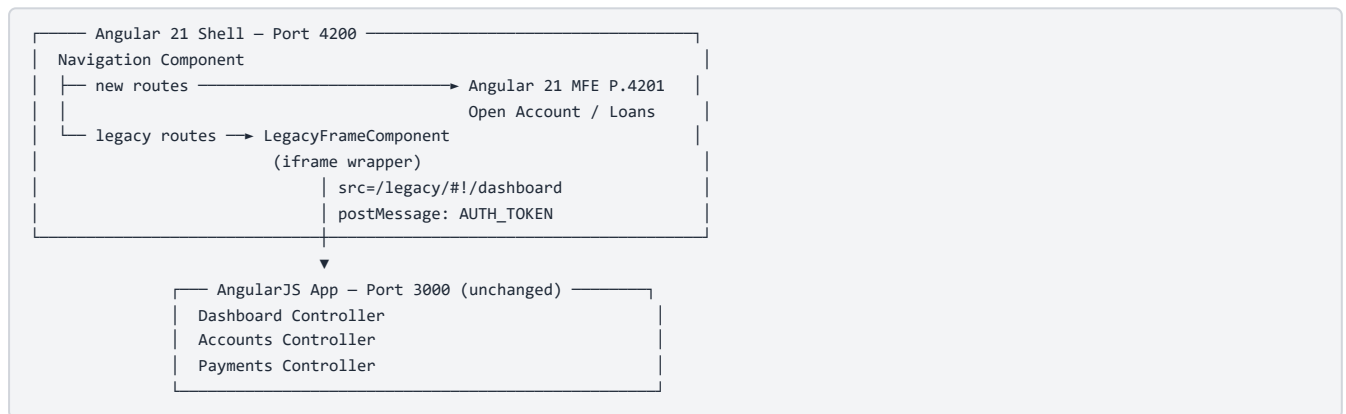
Context: This section is for architects choosing **how** to physically deploy AngularJS and Angular 21 side-by-side during migration. The strategies in section 6.2 describe *what to do*; the topologies below describe *how the apps are wired together in production*.

Overview — Which Topology to Choose

Topology	Legacy Impact	Team Coupling	User Experience	Best For
T1 — Iframe Bridge	Zero changes to AngularJS	None	Slight reflow seam	Teams that cannot touch legacy codebase
T2 — Polyrepo Shell + Remotes	AngularJS served from own repo	Low	Seamless (shell owns chrome)	Large orgs, multiple autonomous teams
T3 — Dual-Tile Dashboard	AngularJS tiles alongside Angular 21	Low	Side-by-side visible progress	Product teams wanting visible migration status
T4 — Split-Route Navigation	AngularJS behind a toggle URL	Low	Per-route choice; user selects	Teams doing rapid per-feature migration with A/B

Topology 1 — Iframe Bridge (Zero-Touch Legacy)

The Angular 21 shell is a thin chrome (header, nav, footer). Every legacy route is served inside an `<iframe>` pointing at the AngularJS app's own URL. No changes whatsoever to the AngularJS codebase.



Deployment:

Production URLs	
<code>https://bank.example.com</code>	→ Angular 21 shell (CDN)
<code>https://bank.example.com/legacy/</code>	→ AngularJS app (same origin, own server)
<code>https://bank.example.com/ng21/</code>	→ Angular 21 MFE remote

Auth token forwarding via postMessage:

```
// legacy-frame.component.ts
@Component({
  selector: 'app-legacy-frame',
  standalone: true,
  changeDetection: ChangeDetectionStrategy.OnPush,
  template: `
    <iframe
      #legacyFrame
      [src]="safeUrl()"
      [title]="frameTitle()"
      width="100%"
      height="100%"
      (load)="forwardAuthToken()"
    ></iframe>
  `,
})
export class LegacyFrameComponent {
  readonly path = input.required<string>();
  readonly frameTitle = input('Legacy application');
  private readonly auth = inject(AuthBridgeService);
  private readonly sanitizer = inject(DomSanitizer);

  @ViewChild('legacyFrame') private frameRef!: ElementRef<HTMLIFrameElement>;

  protected readonly safeUrl = computed(() =>
    this.sanitizer.bypassSecurityTrustResourceUrl(`/legacy/index.html${this.path()}`),
  );

  protected forwardAuthToken(): void {
    const token = this.auth.token();
    if (!token) return;
    // postMessage is safe — same origin only
    this.frameRef.nativeElement.contentWindow?.postMessage(
      { type: 'AUTH_TOKEN', token },
      window.location.origin, // strict same-origin target, never use '*'
    );
  }
}
```

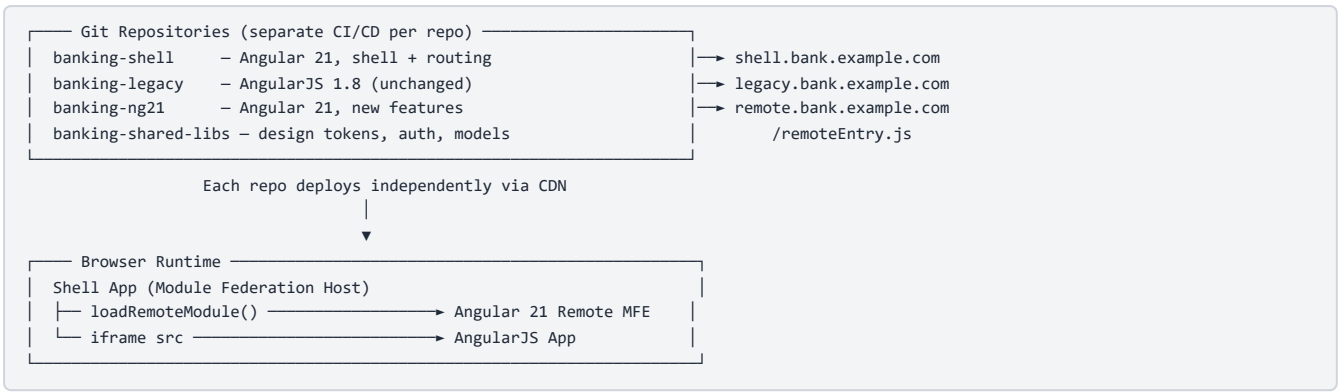
Trade-offs:

Pro	Con
Zero changes to AngularJS codebase	iframe scroll and focus management is brittle
Instant to implement	Deep linking inside iframe requires extra coordination
Easy rollback — remove iframe route	Accessibility: screen readers do not cross iframe boundaries
Full legacy feature parity guaranteed	Both Angular bundles loaded in memory simultaneously

Best for: Teams where the AngularJS codebase is owned by a separate (possibly external) team, or where it is contractually or politically difficult to touch the legacy code.

Topology 2 — Polyrepo with Angular 21 Shell + Module Federation

Each application lives in its own git repository with its own CI/CD pipeline. The Angular 21 shell loads Angular 21 feature remotes via Module Federation dynamic imports, and loads the AngularJS app via iframe for legacy routes. This is the **enterprise standard** for organisations with autonomous product teams.



Shell environment config — remote URLs come from environment, not hardcoded:

```
// environments/environment.ts
export const environment = {
  shellVersion: '1.0.0',
  remotes: {
    legacyBaseUrl: 'https://legacy.bank.example.com',
    accountsMfe: 'https://remote.bank.example.com/accounts/remoteEntry.js',
    loansMfe: 'https://remote.bank.example.com/loans/remoteEntry.js',
    paymentsMfe: 'https://remote.bank.example.com/payments/remoteEntry.js',
  },
  featureFlags: {
    useNewOpenAccount: false, // flip to true when ng21 feature is ready
    useNewDashboard: true,
  },
};

// shell.routes.ts - dynamic remote loading driven by feature flags
export const SHELL_ROUTES: Routes = [
  {
    path: 'open-account',
    loadComponent: () =>
      environment.featureFlags.useNewOpenAccount
        ? loadRemoteModule({
            type: 'module',
            remoteEntry: environment.remotes.accountsMfe,
            exposedModule: './OpenAccountComponent',
          }).then((m) => m.OpenAccountComponent)
        : import('./legacy-frame/legacy-frame.component').then((m) => m.LegacyFrameComponent),
  },
];
```

CI/CD independence — each repo ships on its own cadence:

Release Trains		
banking-shell	→ deploys on nav/routing changes	(rare)
banking-legacy	→ deploys only for legacy bug fixes	(rare)
banking-ng21	→ deploys on every feature completion	(frequent)
banking-shared	→ deploys on design system updates	(occasional)
No team waits for another team's release cycle.		

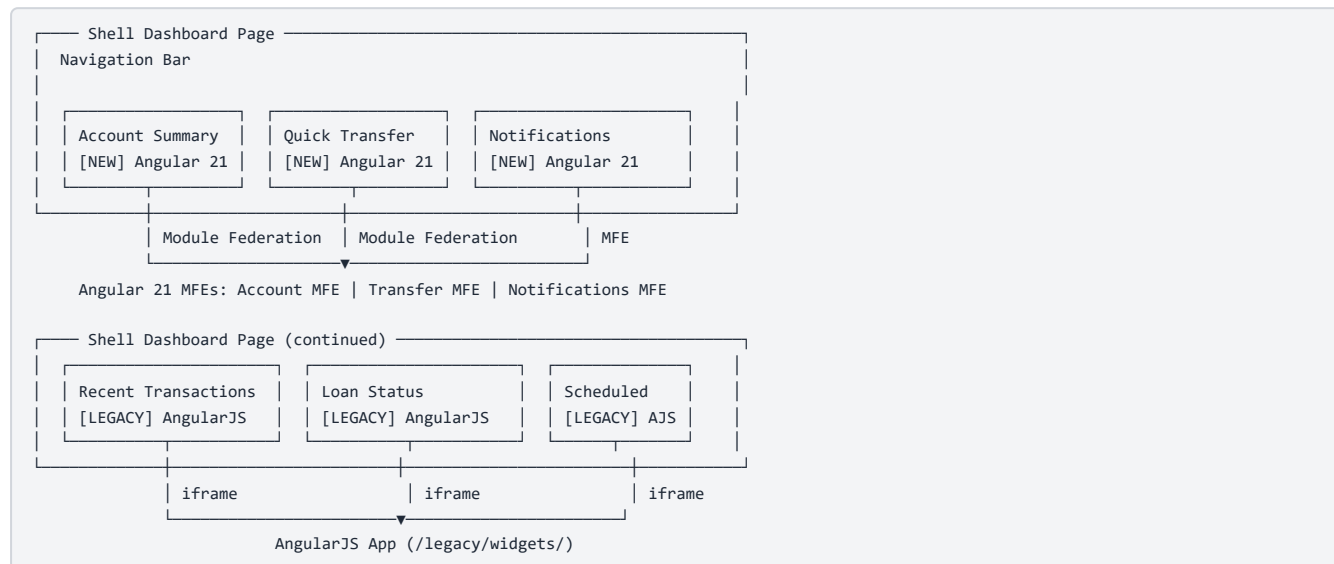
Trade-offs:

Pro	Con
Full team autonomy — each repo has own CI/CD	Shared lib versioning requires discipline
Shell rarely changes — stable chrome for users	Module Federation version drift between remotes
Feature remotes deploy without shell changes	Debugging across repos requires correlated logging
AngularJS team unchanged — zero friction	Initial Nx + MFE setup takes ~2 sprints

Topology 3 — Dual-Experience Tile Dashboard (Side-by-Side Progress)

The shell's home dashboard displays feature **tiles** — each tile is independently served by either the AngularJS app (legacy) or the Angular 21 remote (new). Tiles with a **Legacy** badge use iframe micro-containers; tiles with a **New** badge are Angular 21 components loaded via Module Federation. As features migrate, tiles change badge from amber (Legacy) to green (New) — the migration is literally visible on the product dashboard.

This topology is particularly effective for stakeholder reporting: a product manager can see migration progress without reading a Jira board.



Tile registry — feature flag drives which implementation loads:

```
// dashboard-tile.registry.ts
export type TileStatus = 'legacy' | 'new' | 'migrating';

export interface TileConfig {
  id: string;
  title: string;
  status: TileStatus;
  legacyWidgetPath?: string; // /legacy/widgets/<path>
  mfeRemoteEntry?: string; // Module Federation remote URL
  mfeExposedModule?: string;
}

export const TILE_REGISTRY: TileConfig[] = [
  {
    id: 'account-summary',
    title: 'Account Summary',
    status: 'new',
    mfeRemoteEntry: environment.remotes.accountsMfe,
    mfeExposedModule: './AccountSummaryWidget',
  },
  {
    id: 'recent-transactions',
    title: 'Recent Transactions',
    status: 'legacy',
    legacyWidgetPath: '/widgets/transactions',
  },
  {
    id: 'loan-status',
    title: 'Loan Status',
    status: 'migrating', // In progress – show legacy + migration notice
    legacyWidgetPath: '/widgets/loans',
    mfeRemoteEntry: environment.remotes.loansMfe,
    mfeExposedModule: './LoanStatusWidget',
  },
];

// dashboard-tile.component.ts
@Component({
  selector: 'app-dashboard-tile',
  standalone: true,
  changeDetection: ChangeDetectionStrategy.OnPush,
  imports: [NgComponentOutlet, LegacyFrameComponent],
  template: `
    <div class="tile" [attr.data-status]="tile().status">
      <header class="tile-header">
        <h3>{{ tile().title }}</h3>
        @switch (tile().status) {
          @case ('new') {
            <span class="badge badge-new" aria-label="New Angular 21 experience">NEW</span>
          }
          @case ('legacy') {
            <span class="badge badge-legacy" aria-label="Legacy experience">LEGACY</span>
          }
          @case ('migrating') {
            <span class="badge badge-migrating" aria-label="Migration in progress">
              >IN PROGRESS</span>
            >
          }
        }
      </header>

      <div class="tile-content">
        @if (tile().status === 'legacy') {
          <app-legacy-frame [path]="tile().legacyWidgetPath!" [frameTitle]="tile().title" />
        } @else {
          @defer {
            <ng-container *ngComponentOutlet="lazyComponent()" />
          } @loading {
            <div
              class="tile-skeleton"
              aria-busy="true"
              [attr.aria-label]=" 'Loading ' + tile().title "
            ></div>
          } @error {
            <p class="tile-error" role="alert">{{ tile().title }} is temporarily unavailable.</p>
          }
        }
      </div>
    </div>`
})

```

```

    </div>
  },
  })
export class DashboardTileComponent {
  readonly tile = input.required<TileConfig>();
  protected readonly lazyComponent = signal<Type<unknown> | null>(null);

  constructor() {
    effect(() => {
      const t = this.tile();
      if (t.status !== 'legacy' && t.mfeRemoteEntry && t.mfeExposedModule) {
        loadRemoteModule({
          type: 'module',
          remoteEntry: t.mfeRemoteEntry,
          exposedModule: t.mfeExposedModule,
        }).then((m) => this.lazyComponent.set(m.default));
      }
    });
  }
}

```

Stakeholder migration progress visible in the product itself:

Dashboard Migration Status – Sprint 14

[NEW] ✓	Account Summary	Angular 21	Shipped: Sprint 10
[NEW] ✓	Quick Transfer	Angular 21	Shipped: Sprint 11
[NEW] ✓	Notifications	Angular 21	Shipped: Sprint 12
[IN] ⚙	Loan Status	Migrating	ETA: Sprint 16
[LEG] ⚠	Recent Transactions	AngularJS	ETA: Sprint 17
[LEG] ⚠	Scheduled Payments	AngularJS	ETA: Sprint 18

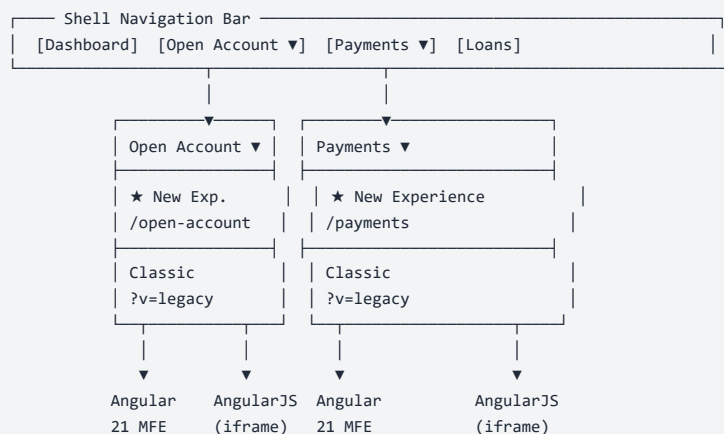
Migration completion: 50% (3 of 6 tiles migrated)

Topology 4 — Split-Route Navigation (Hover Dropdown per Feature)

The shell's navigation bar renders each feature item with a **split-button dropdown chevron**. Hovering or pressing the chevron reveals two options:

- **New Experience** — routes to the Angular 21 version
- **Classic** — routes to the AngularJS version (via iframe)

This gives every user explicit control over their experience. Power users and early adopters self-select into Angular 21. Conservative users stay on the classic route until they are ready — or until the classic option is retired by turning off the feature flag.



User experience — what the navigation looks like:

Navigation Bar

[Dashboard] [Open Account ▼] [Payments ▼] [Statements] [Settings]

▼ (on hover / keyboard focus)

★ New Experience /open-account	← default for new users
Classic /open-account?v=legacy	← legacy AngularJS

User preference is persisted in localStorage so their choice survives navigation and page reloads.

Shell routing — `?v=legacy` query param drives the version toggle:

```
// split-route-host.component.ts - host for any feature with dual routes
@Component({
  selector: 'app-split-route-host',
  standalone: true,
  changeDetection: ChangeDetectionStrategy.OnPush,
  imports: [RouterOutlet, LegacyFrameComponent],
  template: `
    @if (useLegacy()) {
      <app-legacy-frame [path]="legacyPath()" [frameTitle]="featureTitle()" />
    } @else {
      <router-outlet />
    }
  `,
})
export class SplitRouteHostComponent {
  readonly featureTitle = input.required<string>();
  readonly legacyPath = input.required<string>();

  private readonly route = inject(ActivatedRoute);

  // Read ?v=legacy from URL; fallback to localStorage preference
  protected readonly useLegacy = computed(() => {
    const queryV = this.route.snapshot.queryParamMap.get('v');
    if (queryV === 'legacy') return true;
    if (queryV === 'new') return false;
    return localStorage.getItem(`prefer-legacy:${this.featureTitle()}`) === 'true';
  });

  constructor() {
    effect(() => {
      // Persist choice - next visit to this route remembers preference
      localStorage.setItem(`prefer-legacy:${this.featureTitle()}`, String(this.useLegacy()));
    });
  }
}

// split-nav-item.component.ts - navigation item with dual-route dropdown
@Component({
  selector: 'app-split-nav-item',
  standalone: true,
  changeDetection: ChangeDetectionStrategy.OnPush,
  imports: [RouterLink, RouterLinkActive],
  template: `
    <div class="split-nav-item" role="none">
      <!-- Primary link - goes to stored preference or new by default -->
      <a [routerLink]="newRoute()" routerLinkActive="active" class="split-nav-primary">{{
        label()
      }}</a>

      <!-- Chevron opens dropdown -->
      <button
        class="split-nav-chevron"
        [attr.aria-expanded]="dropdownOpen()"
        [attr.aria-label]="'More options for ' + label()'"
        (click)="dropdownOpen.update(v => !v)"
        type="button"
      >
        ▼
      </button>

      @if (dropdownOpen()) {
        <div class="split-nav-dropdown" role="menu">
          <a [routerLink]="newRoute()" role="menuitem" class="split-nav-option--new">
            <span aria-hidden="true">★</span> New Experience
          </a>
          <a [routerLink]="legacyRoute()" role="menuitem" class="split-nav-option--classic">
            Classic
          </a>
        </div>
      }
    </div>
  `,
})
export class SplitNavItemComponent {
  readonly label = input.required<string>();
  readonly newRoute = input.required<string>();
  readonly legacyRoute = input.required<string>();
  protected readonly dropdownOpen = signal(false);
}
```



```

}

// app.routes.ts — wiring the split-route host
export const routes: Routes = [
  {
    path: 'open-account',
    component: SplitRouteHostComponent,
    data: {
      featureTitle: 'open-account',
      legacyPath: '/#!/open-account',
    },
    children: [
      {
        path: '',
        loadComponent: () =>
          loadRemoteModule({
            type: 'module',
            remoteEntry: environment.remotes.accountsMfe,
            exposedModule: './OpenAccountComponent',
          }).then((m) => m.OpenAccountComponent),
      },
    ],
  },
];

```

Deep linking — the two URL forms:

/open-account → Angular 21 version (new experience, default)
 /open-account?v=legacy → AngularJS version inside iframe
 /open-account?v=new → Force Angular 21 (for support teams to share)

The feature team controls the default by setting useNewFeature in environment.ts.
 The legacy URL always remains available until AngularJS is decommissioned.

When to retire the Classic option:

Classic option removal criteria (agree with product before migration starts):

- ✓ New experience has been live ≥ 4 sprints with no P1/P2 incidents
- ✓ Error rate on new route $\leq$ baseline error rate
- ✓ WCAG AA compliance confirmed via automated + manual audit
- ✓ Classic route traffic $< 5\%$ of total route traffic for 2 consecutive sprints
- ✓ Product owner sign-off documented as ADR update

Once all criteria met → remove ?v=legacy handling → retire AngularJS route

Trade-offs:

Pro	Con
Users always have a fallback route during migration	Two URLs per feature adds routing complexity
A/B testing built in — analytics per URL variant	Navigation component needs split-nav logic
Deep linking works for both versions	Legacy path must stay stable until decommission
Rollback is per-route, not per-app	Users may be confused by two options initially

Topology Selection Flowchart — Final Decision Aid

TOPOLOGY SELECTION FLOWCHART

Can you modify the AngularJS codebase?

— NO —————→ Topology 1: iframe Bridge
(zero AngularJS changes needed)

— YES



Are product teams autonomous per feature domain?

— YES (separate teams per domain) —→ Topology 2: Polyrepo Shell + Remotes
(independent CI/CD per team)

— NO (single team owns migration)



Do stakeholders need visible migration progress?

— YES (product/C-suite visibility) —→ Topology 3: Dual-Tile Dashboard

— NO (developer-facing only) —————→ Topology 4: Split-Route Navigation

Architect's recommendation: For most enterprise banking migrations, combine **Topology 2** (Polyrepo with Module Federation for team autonomy) with **Topology 4** (Split-Route Navigation for user-facing rollback safety). Use Topology 3 when you have a C-suite sponsor who needs to see migration progress visualised without reading a Jira board.

Recommended Migration Architecture (Best Way)

RECOMMENDED: NX + AI-ASSISTED + FEATURE FLAGS

MONTH 1-2: Foundation

- ✓ Create Nx workspace with shell, banking-legacy, banking-ng21
- ✓ Module Federation wired: shell is host, ng21 is remote
- ✓ Shared libs: models, auth, UI tokens
- ✓ Run automated codemods (ng update, control-flow migration)
- ✓ Feature flag service in shell (LaunchDarkly / custom signal)

MONTH 2-4: AI-Powered Feature Migration

- ✓ Copilot converts controllers → standalone components
- ✓ Copilot converts services → @Injectable with signals
- ✓ Copilot generates Vitest specs for every migrated file
- ✓ Each migrated feature toggled ON for internal staff first
- ✓ Shell toggle visible to users who opt in to "New Experience"

MONTH 4-6: Cutover

- ✓ New customers default to Angular 21 experience
- ✓ Existing customers opt-in (or auto-migrated in waves)
- ✓ Monitoring: compare error rates old vs new per feature
- ✓ Kill switch: one signal flip rolls everyone back to legacy

MONTH 6+: Decommission

- ✓ 100% traffic on Angular 21
- ✓ Remove banking-legacy app from Nx workspace
- ✓ Remove iframe bridge, legacy-frame component
- ✓ Remove @angular/upgrade (was never needed in this approach)
- ✓ Remove AngularJS bundle – save 100-150KB from initial load

Complete Migration Checklist

Task	Strategy	Angular Version	Status
Foundation			
Create Nx workspace with shell + legacy + ng21 apps	Nx MFE	—	☐
Configure Module Federation (host + remote)	Nx MFE	17+	☐
Create shared-models, shared-auth, shared-ui libs	All	—	☐
Add feature flag service to shell	Nx MFE	Any	☐
Run <code>ng update automated</code> codemods	All	21	☐
Run <code>@angular/codemod control-flow</code> migration	All	17+	☐
AI-Assisted Conversion			
Catalogue all controllers, services, directives	AI	—	☐
Copilot-convert controllers → standalone components	AI	21	☐
Copilot-convert services → @Injectable + signals	AI	21	☐
Copilot-convert directives → components or pipes	AI	21	☐
Copilot-generate Vitest specs (≥80% coverage gate)	AI	—	☐
Modern Angular 21 Patterns			
Replace <code>\$scope</code> → <code>signal()</code>	All	16+	☐
Replace <code>\$watch</code> → <code>computed()</code> / <code>effect()</code>	All	16+	☐
Replace <code>ngModel</code> → signal forms	All	21	☐
Replace <code>*ngIf</code> / <code>*ngFor</code> → <code>@if</code> / <code>@for</code>	All	17+	☐
Replace <code>@Input()</code> / <code>@Output()</code> → <code>input()</code> / <code>output()</code>	All	17+	☐
Replace constructor injection → <code>inject()</code>	All	14+	☐
Add <code>@defer</code> blocks for non-critical UI	All	17+	☐
Enable <code>ChangeDetectionStrategy.OnPush</code> on all components	All	Any	☐
Cutover & Cleanup			
Enable feature flags for internal staff → beta → all users	Nx MFE	—	☐
Enable SSR + hydration	All	17+	☐
Remove iframe bridge and legacy-frame component	Nx MFE	—	☐
Remove banking-legacy app from Nx workspace	Nx MFE	—	☐
Remove <code>@angular/upgrade</code> (if used)	ngUpgrade	—	☐
Remove AngularJS bundle (saves ~100–150KB)	All	—	☐

7. Resources

Resource	URL	What it covers
Angular Official Docs	https://angular.dev	Latest Angular 21 APIs
NgRx Docs	https://ngrx.io	State management
Module Federation	https://webpack.js.org/concepts/module-federation	MFE setup
@angular-architects/mf	https://github.com/angular-architects/module-federation-plugin	Angular MFE toolkit
Brad Frost Atomic Design	https://atomicdesign.bradfrost.com	Design system structure
C4 Model	https://c4model.com	Architecture diagrams
12-Factor App	https://12factor.net	App methodology
Refactoring Guru	https://refactoring.guru/design-patterns	GoF patterns with examples
NgRx Signal Store	https://ngrx.io/guide/signals	Signal-based state
Mermaid Diagrams	https://mermaid.js.org	Code-based diagramming
ADR GitHub	https://github.com/joelparkerhenderson/architecture-decision-record	ADR templates
Strangler Fig Pattern	https://martinfowler.com/bliki/StranglerFigApplication.html	Migration strategy